

OOPJ Complete Revision Notes

Shivam Zanjurne · One-shot revision of Day 1 – Day 15 (7 Sep – 23 Sep 2026) · Concepts + runnable code

Table of Contents

1. Programming Fundamentals
2. Java Architecture and Basics
3. Objects, Memory and Constructors
4. Logic Design: The Date Class
5. Inheritance, Casting and Polymorphism
6. final, abstract, sealed and Packages
7. Strings, Object Class, Wrappers and GC
8. Exception Handling
9. File I/O and Serialization
10. Interfaces
11. Collections Framework
12. Functional Interfaces and Lambdas
13. Stream API
14. Generics and Wildcards
15. Multithreading and Synchronization
16. Date and Time API
17. Interview Traps and Final Checklist

Programming Fundamentals

Day 1–2. Everything in Java makes more sense once you know what a compiler, linker and loader actually do. These notes start at the C/C++ level and then show how Java changes the picture.

The 5 essentials of a development environment

#	Component	Job	Example
1	Text editor	Write source code	Notepad++, VS Code
2	Compiler	Source → object/bytecode; catches syntax errors	<code>gcc</code> , <code>javac</code>
3	Linker	Joins object files + libraries into one executable	<code>ld</code> , MSVC linker
4	Libraries	Pre-compiled reusable code	<code>libc</code> , <code>rt.jar</code> / JDK modules
5	Debugger	Step through running code, inspect memory	<code>gdb</code> , Eclipse debugger

An **IDE** (Eclipse, IntelliJ, Visual Studio) simply bundles all five behind one window.

The compilation pipeline (C/C++)

```
source.c → Preprocessor → Compiler → Assembler → Linker → program.exe
           (# directives)  (syntax,  (.obj /  (attach libs,
           textual         optimize,  .o)      resolve calls,
           substitution)   codegen)      add COFF header)
```

1. **Preprocessing** – handles every line starting with `#` (`#include` , `#define`). It is *pure text substitution*: `#include <stdio.h>` literally pastes the header's text into your file.
2. **Syntax / grammar check** – the compiler verifies the code against the language grammar. Errors here are *compile-time errors*.
3. **Optimization** – dead-code removal, constant folding, loop unrolling, etc.
4. **Code generation** – produces machine-specific object code (`.obj` on Windows, `.o` on Linux).

Header files vs libraries

- **Header (`.h`)** – contains only **declarations** (function prototypes, macros, type definitions). Tells the compiler "*this function exists somewhere, here is its signature*".
- **Library (`.lib` / `.a` / `.dll` / `.so`)** – contains the **compiled definitions** (actual machine code).
- The compiler is happy with just the header; it is the **linker** that must find the definition in a library. A "missing header" is a compile error; an "unresolved external symbol" is a **link** error.

What the linker does

1. Attaches the required libraries to your object code.
2. Resolves every function *call* to its actual *definition* (address).
3. Adds the **COFF header** (Common Object File Format) – a Windows-specific header describing how the OS should load the executable. This is exactly *why an `.exe` compiled on Windows will not run on Linux* (Linux expects ELF).
4. Sets `main` as the default **entry point** of the program.

Dangling pointer (C/C++ problem that Java eliminates)

```
int* getValue() {  
    int local = 42;           // lives on the stack frame of getValue()  
    return &local;           // frame is destroyed on return → address is now garbage  
}
```

// caller's pointer is DANGLING

- Java never gives you a raw address, and objects live on the **heap** managed by the GC, so a reference can never "dangle" – it is either valid or `null`.

Pointer size

- A pointer's size depends on the **address-bus width** of the machine (4 bytes on 32-bit, 8 bytes on 64-bit), **not** on the type it points to. `char*`, `int*`, `double*` are all the same size.

OOP: the 5 major features

#	Feature	One-line meaning	Real-world hook
1	Abstraction	<i>Generalization</i> – extract what is common into a parent type	Sedan, SUV → <code>PassengerCar</code>
2	Encapsulation + Hiding	Bind data and behavior into one unit; then hide the data	Capsule: medicine inside, casing outside
3	Inheritance	<i>Specialization</i> – extend an existing type with more detail	<code>PassengerCar</code> → <code>Sedan</code>
4	Polymorphism	One name, many forms	<code>draw()</code> behaves differently for Circle/Square
5	Modularity	Independent, replaceable, reusable modules	Car parts from different vendors

1. Abstraction = generalization

- Move upward in the hierarchy: identify the shared properties/behaviors of many concrete things and put them into a general type.
- Result: the abstract type (`PassengerCar`) is *conceptual* – you never buy "a passenger car", you buy a specific model.

2. Encapsulation and hiding

- Encapsulation:** putting data (fields) and behavior (methods) *together* inside one unit – the class.
- Hiding:** making the data `private` so it is only reachable through methods.
- Key exam point:** *hiding is impossible without encapsulation* – you cannot hide data behind methods unless data and methods live in the same unit.

```

class Account {
    private double balance;           // hidden

    public void deposit(double amount) { // controlled access
        if (amount > 0) balance += amount;
    }
    public double getBalance() { return balance; }
}

```

3. Inheritance = specialization

- Move downward: take an existing type and add specifics.
- Types of inheritance: **single**, **multi-level**, **multiple** (C++ only for classes; Java via interfaces), **hierarchical**, **hybrid**.

4. Polymorphism

Kind	Also called	Resolved	Mechanism
Static	Compile-time / early binding	At compile time	Overloading – compiler <i>mangles</i> the name using <code>name + argument types</code> , so <code>add(int,int)</code> and <code>add(float,float)</code> become two different internal names
Dynamic	Runtime / late binding	At run time	Overriding – needs <code>virtual</code> in C++; default in Java (every non-static, non-final, non-private method is virtual)

5. Modularity

- Split the system into independent modules with clean interfaces so each can be developed, tested and replaced separately. Java packages and (Java 9+) modules implement this.

Minor features

- **Persistence / Serialization** – storing object state beyond program life. *Persistence* = data only; *Serialization* = data + **metadata** (class name, field types).
- **Concurrency** – multiple threads of execution (Section 15).
- **RTTI / RTCI / Reflection** – Run-Time Type Identification (`instanceof`), Run-Time Class Information (`getClass()`), and Reflection (inspecting/invoking members by name at runtime).

Object – the exam definition

An **object** is a real-time entity that has an **identity** (unique address/name), a **state** (properties/fields) and **behaviors** (methods).

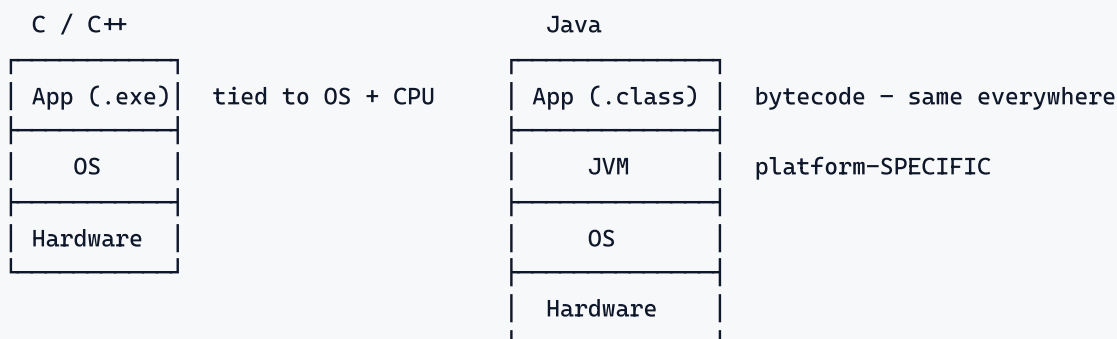
Java Architecture and Basics

Day 2–3. Why Java exists, how "write once, run anywhere" is achieved, and the first programs.

History in one glance

- Created by **James Gosling** at **Sun Microsystems** (acquired by Oracle in 2010).
- Original name **Oak** (after a tree outside his office), later renamed Java.
- First target was **set-top boxes** / embedded devices – hence the emphasis on portability and safety.
- **Java 1.0 released in 1994–95**; Java 2 (1998) brought the JIT compiler, Swing and Collections.

The architecture: why it is platform independent



- **App → JVM → OS → Hardware.** The JVM absorbs all OS/CPU differences.
- **"Write once, run anywhere"** – the *bytecode* is portable; a *different JVM* is installed per platform.

Compilation and execution flow

```
Program.java —javac→ Program.class —java→ JVM interprets / JIT-compiles → output
              (compiler)  (bytecode)   (launcher)
```

- `.java` → `javac` → `.class` (bytecode, platform-independent) → **JVM** (platform-specific) executes it.
- **Java 1.0 problem:** the JVM interpreted bytecode *line by line*, every time – slow.
- **Java 2 fix – JIT (Just-In-Time) compiler:** hot bytecode is converted to native machine code **once**, cached, and reused on later calls. Modern HotSpot JVMs mix interpretation (for cold code) and JIT (for hot code).

JDK vs JRE vs JVM

Term	Contains	Who needs it
JVM	The engine that executes bytecode	Everyone (part of JRE)
JRE	JVM + core libraries (<code>java.lang</code> , <code>java.util</code> ...)	End users – execute only
JDK	JRE + <code>javac</code> , <code>javadoc</code> , <code>jdb</code> , <code>jar</code> ...	Developers – develop + execute + debug

Java editions

Edition	Full name	Purpose	Status
JSE	Java Standard Edition	Core language + libraries (this course)	Active
JEE / Jakarta EE	Enterprise Edition	Distributed n-tier apps: Servlets, JSP, EJB, JPA, JMS	Active (now "Jakarta EE")
JME	Micro Edition	Handheld / feature phones	Largely dead (replaced by Android)

Data types – fixed sizes everywhere

Type	Size	Range / notes	Default (as field)
<code>byte</code>	1 byte	–128 to 127	<code>0</code>
<code>short</code>	2 bytes	–32,768 to 32,767	<code>0</code>
<code>int</code>	4 bytes	≈ ±2.1 billion	<code>0</code>
<code>long</code>	8 bytes	≈ ±9.2 × 10 ¹⁸ (literal needs <code>L</code> : <code>10L</code>)	<code>0L</code>
<code>float</code>	4 bytes	~7 decimal digits (literal needs <code>f</code> : <code>3.14f</code>)	<code>0.0f</code>
<code>double</code>	8 bytes	~15 decimal digits	<code>0.0</code>
<code>boolean</code>	1 bit (JVM-dependent)	<code>true</code> / <code>false</code> only – not 0/1	<code>false</code>
<code>char</code>	2 bytes	Unicode (0–65,535) vs 1-byte ASCII in C	<code>'\u0000'</code>

- In C, `int` is 2 bytes on a 16-bit compiler and 4 on 32-bit – code behaves differently. Java **fixes** sizes so that bytecode behaves identically on every JVM.
- **Java is not 100% object-oriented** because primitives are *not* objects (Smalltalk is 100% OOP). Primitives exist purely for performance.

Wrapper classes

Primitive	Wrapper	Parse from string
byte	Byte	Byte.parseByte(s)
short	Short	Short.parseShort(s)
int	Integer	Integer.parseInt(s)
long	Long	Long.parseLong(s)
float	Float	Float.parseFloat(s)
double	Double	Double.parseDouble(s)
char	Character	–
boolean	Boolean	Boolean.parseBoolean(s)

```
int num = Integer.parseInt("123");           // String → int
String str = Integer.toString(123);          // int → String
Integer boxed = Integer.valueOf(123);         // int → Integer object
int maxInt = Integer.MAX_VALUE;              // useful constants
```

Java references vs C pointers

- A Java **reference** is an address internally – but:
 - **No pointer arithmetic:** `ref++`, `ref + 1` are compile errors (security – you can't walk memory).
 - **No `&` / `*` operators:** you never see or dereference the raw address.
 - **Not** a C++ alias reference (`int& r = x;`) – Java references can be reassigned and can be `null`.

Hello World and `main`

```
class Program {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Each word in the signature matters:

Keyword	Why it is there
<code>public</code>	JVM (outside the class/package) must be able to call it
<code>static</code>	JVM calls <code>main</code> on the class without first creating an object – there is no object yet
<code>void</code>	Return value is not used by the JVM (use <code>System.exit(code)</code> for exit codes)
<code>String[] args</code>	Command-line arguments (<code>java Program a b</code> → <code>args = {"a","b"}</code>)

- If the class is declared `public`, the **file name must match** the class name exactly (`Program.java`).
- Compile: `javac Program.java` → produces `Program.class`. Run: `java Program` (no `.class` extension).
- One `.java` file can hold many classes → one `.class` file per class.

Console input – the pre- `Scanner` pattern

`System.in` is a raw byte stream, so reading a number means: read bytes → strip the line terminator → build a `String` → parse.

```
import java.io.IOException;

class Program {
    public static void main(String[] args) throws IOException {
        byte[] arrInput = new byte[100];
        int length = System.in.read(arrInput);           // includes '\r' and '\n' on Windows

        byte[] arrFinal = new byte[length - 2];         // strip "\r\n" (use -1 on Linux)
        System.arraycopy(arrInput, 0, arrFinal, 0, length - 2);

        String objString = new String(arrFinal);        // bytes → String
        int num = Integer.parseInt(objString);          // String → int
        System.out.println("You entered " + num);
    }
}
```

- `System.arraycopy(src, srcPos, dest, destPos, length)` – native, fast array copy.
- Today you use `Scanner` (Section 13) but the byte-level version shows what is really happening.

Design rule: no console I/O inside business classes

Business-logic classes (`Date`, `Employee`, `Account` ...) must **never** call `System.out.println` or read from `System.in`. Put I/O only in `main()` or a dedicated `ConsoleInput` / view class.

Why: the same `Employee` class should work unchanged in a console app, a Swing GUI, a web servlet or a REST API. Console I/O baked into the class kills that reuse.

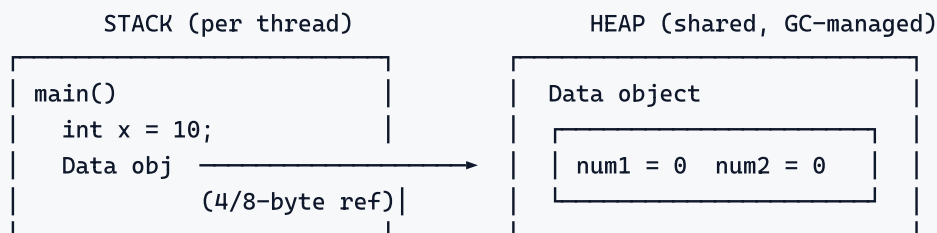
```
// Good: class exposes data, caller decides how to show it
class Employee {
    private String name;
    public String getName() { return name; }
}

class ConsoleInput { // dedicated I/O helper
    static int readInt(String prompt) {
        System.out.print(prompt);
        return new java.util.Scanner(System.in).nextInt();
    }
}
```

Objects, Memory and Constructors

Day 4. Where things live, in what order they are initialized, and how `static` changes the rules.

Stack vs heap



- **Primitives as local variables** → stack. **Objects** → *always* on the heap via `new`. There is no "stack object" in Java (unlike C++ `Data d;`).
- `Data obj;` creates **only a reference** (4 bytes on 32-bit JVM) on the stack – **no object** yet.
- `Data obj = new Data();` creates reference + heap object; the reference stores the heap address.

```

class Data {
    int num1;
    int num2;
}

class Program {
    public static void main(String[] args) {
        Data obj1;           // reference only - using obj1 here is a compile error
        Data obj2 = null;     // reference explicitly pointing nowhere
        obj2.num1 = 1;        // RUNTIME: NullPointerException
        Data obj3 = new Data(); // reference + object
        obj3.num1 = 1;        // OK
    }
}

```

Default values on the heap

Field type	Default
Numeric (byte ... double)	0 / 0.0
boolean	false
char	'\u0000' (blank)
Reference (any object, String , array)	null

Local variables have no defaults – using an uninitialized local is a compile error. Only **fields** (heap) get defaults.

Three places to initialize an instance variable

#	Location	Use when	Example
1	At declaration	Value is a fixed literal	<code>int count = 0;</code>
2	Initialization block { ... }	Value needs logic/loops	Fill a lookup array
3	Constructor	Value comes per object from the caller	<code>this.name = name;</code>

These are *alternatives* for a given variable – pick one. Execution order **for every new** :

field declarations (in order) → init blocks (in order) → constructor body

```

class Order {
    private int id = 100;                // 1. declaration

    {                                    // 2. init block
        System.out.println("Init block, id = " + id);
        id = id * 2;
    }

    Order() {                            // 3. constructor
        System.out.println("Constructor, id = " + id);
    }

    public static void main(String[] args) {
        new Order();
        // Output:
        // Init block, id = 100
        // Constructor, id = 200
    }
}

```

static – class-level members

- **Static variable:** **one copy per class**, shared by all objects; lives in the method area/metaspace, not inside any object. Access as `ClassName.var` (never through an object, even though Java allows it).
- **Static block** `static { ... }`: runs **once**, when the class is **loaded**; may touch **only static members** (there is no `this` yet).
- **Static method:** callable without an object; cannot use `this` or instance members directly.

```

class Counter {
    static int totalObjects;             // shared
    int id;                             // per object

    static {
        totalObjects = 0;
        System.out.println("Class loaded"); // printed exactly once
    }

    Counter() {
        totalObjects++;
        id = totalObjects;
    }

    public static void main(String[] args) {
        Counter a = new Counter();
        Counter b = new Counter();
        System.out.println(Counter.totalObjects); // 2
        System.out.println(a.id + " " + b.id);    // 1 2
    }
}

```

Constructor rules

1. **Same name as the class, no return type** – not even `void`.
 - `public void Data() { }` compiles but is an ordinary **method** that happens to share the class name. Classic trap.
2. Invoked **automatically on every new** – you cannot call it like a method later.
3. If you write **no** constructor, the compiler supplies a **default no-arg** constructor. If you write **any** constructor, the default is **not** supplied – `new Data()` becomes a compile error unless you add it back.
4. **Overloadable** – constructors may differ by **number, type or sequence** of parameters.
5. `this( ... )` calls another constructor of the same class (must be the first statement).

```
class Data {  
    private int num1;  
    private int num2;  
  
    Data() {                                // no-arg  
        this(0, 0);                        // delegate to the 2-arg version  
    }  
    Data(int num1) {                       // 1-arg  
        this(num1, 0);  
    }  
    Data(int num1, int num2) {             // 2-arg - single source of truth  
        this.num1 = num1;                 // 'this' disambiguates field vs parameter  
        this.num2 = num2;  
    }  
}
```

Control flow differences from C/C++

- `if` requires a **boolean**: `if (num = 10)` is a **compile error** in Java (assignment yields `int`, not `boolean`). In C++ it silently compiles and is always true – Java removes a whole class of bugs.
- No implicit int → boolean: `while (1)` is illegal; write `while (true)`.
- `switch` works on `byte/short/char/int`, `String`, enums (and any type with pattern matching in Java 21).

Arrays

- Arrays are **objects on the heap**, have a fixed `length` field (not a method), are **bounds-checked** at runtime (`ArrayIndexOutOfBoundsException`).

```

int[] a = new int[5];           // 1. size only - all elements default to 0
int[] b = new int[]{1, 2, 3, 4, 5}; // 2. size inferred from initializer
int[] c = {1, 2, 3, 4, 5};      // 3. shorthand - ONLY at declaration

for (int i = 0; i < c.length; i++) // classic index loop (need i for position)
    System.out.println(c[i]);

for (int data : c)                // enhanced for - read-only traversal
    System.out.println(data);

```

- `int a[]` also works (C style) but `int[] a` is the Java convention – the type is "int array".
- Enhanced `for` cannot modify the array elements through `data`, and gives no index.

Logic Design: The Date Class

Day 5. A complete, defensively designed class. The lessons here (lookup tables, one source of truth, validation in constructors) are reused in every later program.

Design lessons first

1. **Replace if-else chains with a lookup array.** Instead of 12 `if (month == 1) ... else if (month == 2) ...`, index into `numberOfDays[month]`. Index `0` is a dummy so that month `1` maps to index `1` – readability wins over one wasted `int`.
2. **Single source of truth.** `addDays` is the *only* method that knows how to roll over months and years. `addMonths` and `addYears` compute a day count and **funnel into** `addDays`. One bug-fix location.
3. **Validate in the constructor.** A parameterized constructor that calls `setDate( ... )` ensures an object can *never* exist in an invalid state (day 32, month 13).
4. **Naming discipline.** `addDays`, not `newDD`. Loop variables should say what they are (`monthIndex`), not `i`. **No console I/O** inside the class (Section 2).
5. **Leap years matter.** February's length is data that *changes* – recompute it whenever the year changes.

Complete Date class

```
public class Date {
    private int day;
    private int month;
    private int year;

    // index 0 is a dummy so that month N → numberOfDays[N]
    private final int[] numberOfDays = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    public Date(int day, int month, int year) {
        setDate(day, month, year);           // validation happens on every construction
    }

    private boolean isLeapYear() {
        // divisible by 4, and either divisible by 400 or NOT divisible by 100
        return year % 4 == 0 && (year % 400 == 0 || year % 100 != 0);
    }

    private void refreshFebruary() {
        numberOfDays[2] = isLeapYear() ? 29 : 28;
    }

    public void setDate(int day, int month, int year) {
        if (month < 1 || month > 12)
            throw new IllegalArgumentException("Invalid month: " + month);
        this.year = year;
        refreshFebruary();
        if (day < 1 || day > numberOfDays[month])
            throw new IllegalArgumentException("Invalid day: " + day);
        this.month = month;
        this.day = day;
    }

    public void addDays(int daysToAdd) {
        int totalDays = day + daysToAdd;
        while (totalDays > numberOfDays[month]) {
            totalDays -= numberOfDays[month];    // consume the current month
            month++;
            if (month > 12) {
                month = 1;
                year++;
                refreshFebruary();                // new year → Feb length may change
            }
        }
        day = totalDays;
    }

    public void addMonths(int monthsToAdd) {
        // sum the lengths of the months we will traverse, then delegate
        int sum = 0;
        int tempMonth = month;
        int tempYear = year;
        for (int count = 0; count < monthsToAdd; count++) {
```

```

        sum += numberOfDays[tempMonth];
        tempMonth++;
        if (tempMonth > 12) {
            tempMonth = 1;
            tempYear++;
            numberOfDays[2] = (tempYear % 4 == 0 && (tempYear % 400 == 0 || tempYear %
100 != 0)) ? 29 : 28;
        }
    }
    refreshFebruary(); // restore for the real current year
    addDays(sum);
}

public void addYears(int yearsToAdd) {
    addMonths(yearsToAdd * 12); // funnels through addMonths → addDays
}

public int getDay() { return day; }
public int getMonth() { return month; }
public int getYear() { return year; }

@Override
public String toString() { // caller prints it; the class never does
    return String.format("%02d/%02d/%04d", day, month, year);
}
}

```

Leap-year truth table

Year	% 4 == 0	% 100 != 0	% 400 == 0	Leap?
2024	yes	yes	no	Yes
1900	yes	no	no	No (century, not divisible by 400)
2000	yes	no	yes	Yes (divisible by 400)
2023	no	–	–	No

Traced examples

```

Date d1 = new Date(15, 9, 2026);
d1.addDays(100); // 15 Sep + 15 → 30 Sep, +31 Oct, +30 Nov = 76; 24 left →
24/12/2026

Date d2 = new Date(29, 1, 2024);
d2.addMonths(1); // Jan has 31 days → 29 + 31 = 60 → 60 - 31 = 29 Feb (leap) →
29/02/2024

Date d3 = new Date(29, 1, 2023);
d3.addMonths(1); // 60 - 31 = 29 > 28 (Feb 2023) → 29 - 28 = 1 → 01/03/2023

```

Why the difference? Same code, different data – `numberOfDays[2]` is 29 in 2024 and 28 in 2023. This is exactly why the table must be refreshed when the year changes.

Inheritance, Casting and Polymorphism

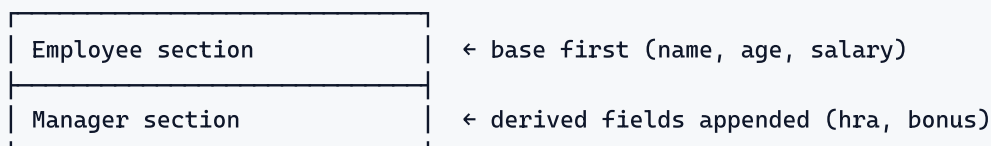
Day 5–6. How derived objects are laid out, why constructors run base-first, how `instanceof` and virtual dispatch really work.

Inheritance basics

- Keyword: `extends`. A class can extend **exactly one** class (Java has single inheritance of classes).
- **Multiple inheritance of classes is not supported** – if two parents both define `display()`, which one does the child get? This *ambiguity* (the diamond problem) is why Java forbids it. Interfaces (Section 10) provide the safe alternative.
- `Object` is the implicit root of every class.

Memory layout of a derived object

Manager object on the heap



- A derived object = **full base section + derived section**. This is why an `Employee` reference can legally point at a `Manager` – the `Employee` part is physically there.

Constructor chaining and `super`

- Order: **base constructor first, then derived** – the base section must be initialized before the derived code can rely on it.
- `super(args)` **must be the first statement** in a derived constructor.
- If you write no `super( ... )`, the compiler inserts `super()` – which **fails to compile** if the base has only *parameterized* constructors. Then `super( ... )` is **mandatory**.
- `super.method()` calls the parent's version (used to *extend* rather than replace behavior).

```

class Employee {
    protected String name;
    protected double salary;

    Employee(String name, double salary) {           // no default constructor!
        this.name = name;
        this.salary = salary;
        System.out.println("Employee constructor");
    }

    public double getSalary() { return salary; }

    @Override
    public String toString() { return name + " earns " + getSalary(); }
}

class Manager extends Employee {
    private double hra;

    Manager(String name, double salary, double hra) {
        super(name, salary);                         // MANDATORY - must be first
        this.hra = hra;
        System.out.println("Manager constructor");
    }

    public double getHRA() { return hra; }

    @Override
    public double getSalary() {                      // overriding
        return super.getSalary() + hra;              // extend, don't replace
    }
}

```

Upcasting and downcasting

```

Employee e = new Manager("Asha", 50000, 10000);    // UPCAST: implicit, always safe
// e.getHRA();                                     compile error - Employee has no getHRA

Manager m = (Manager) e;                           // DOWNCAST: explicit; JVM verifies at
runtime                                             // 10000.0
System.out.println(m.getHRA());

Employee plain = new Employee("Ravi", 30000);
Manager bad = (Manager) plain;                     // RUNTIME: ClassCastException

```

Direction	Syntax	Checked	Visible members
Upcast (child → parent ref)	implicit	compile time	Only the parent's members (child members hidden, but child <i>overrides</i> still run)
Downcast (parent ref → child)	explicit <code>(Child)</code>	runtime – <code>ClassCastException</code> if the real object isn't a <code>Child</code>	Full child API

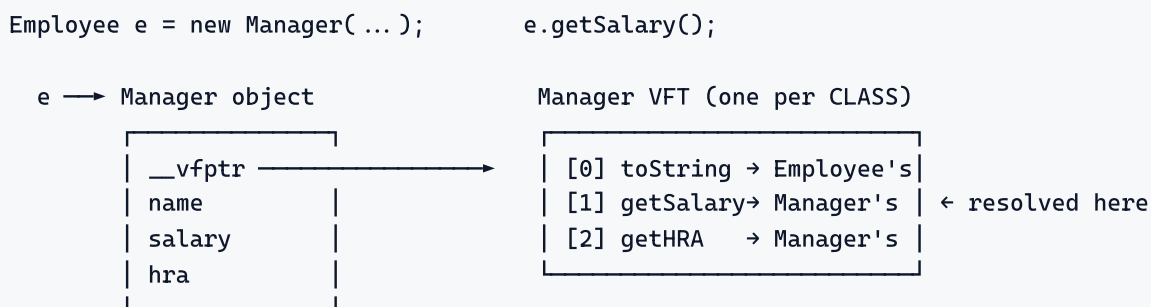
Safe downcasting with `instanceof`

```
// Classic (all versions)
if (e instanceof Manager) {
    Manager mm = (Manager) e;
    System.out.println(mm.getHRA());
}

// Pattern matching (Java 16+) – test + cast + bind in one step
if (e instanceof Manager mm) {
    System.out.println(mm.getHRA());
}
```

- `instanceof` is Java's **RTTI**. Internally it compares the object's hidden **VTable pointer** (class metadata pointer) against the target class hierarchy.
- `null instanceof Anything` is always `false` – no NPE.

Virtual function table (VFT) internals



- Each **object** carries one hidden pointer (`__vfptr` , 4/8 bytes); each **class** has one **VFT** holding the addresses of its virtual methods (inherited slots point to the parent's code unless overridden).
- A call is resolved at runtime as `object → __vfptr → table[slotOffset]` – the **VFT of the actual object** decides which method runs, regardless of the reference type. This *is* dynamic polymorphism.
- **C++**: only methods marked `virtual` go into the table. **Java**: **all** instance methods are virtual by default (except `private` , `static` , `final`) – no keyword needed, same mechanism.

Overriding vs overloading vs hiding

	Overriding	Overloading	Hiding (C++ only)
Signature	Same name + parameters	Same name, different parameters	Same name, non-virtual
Needs inheritance	Yes	No (same scope)	Yes
Resolved	Runtime (VFT)	Compile time (name mangling)	Compile time by pointer type
Return type	Same or covariant subtype	Anything	–
Access	Cannot be narrower	Anything	–
Java annotation	<code>@Override</code>	–	n/a (Java has no method hiding for instance methods)

- Always write `@Override` : if you misspell `toString()` or get a parameter wrong, the compiler reports an error instead of silently creating a new overload.
- Static methods are **not** overridden – a same-signature static method in the child *hides* the parent's (resolved by reference type). Avoid relying on it.

Java's passing semantics

Java is **strictly pass-by-value**. For objects, the *value* being copied is the **reference** (address). There is no true pass-by-reference (no C++ `&`).

```

class Data { int num; }

class Program {
    static void changeField(Data d) {
        d.num = 99;           // follows the copied address → modifies the SAME object
    }
    static void reassign(Data d) {
        d = new Data();       // rebinds the LOCAL copy only
        d.num = 500;          // caller's object untouched
    }
    static void primitive(int n) {
        n = 42;               // copy of the int; caller unaffected
    }

    public static void main(String[] args) {
        Data obj = new Data();
        changeField(obj);     System.out.println(obj.num); // 99 - visible
        reassign(obj);        System.out.println(obj.num); // 99 - reassignment NOT visible
        int x = 1;
        primitive(x);         System.out.println(x);       // 1
    }
}

```

- **Consequence:** you cannot write a `swap(a, b)` that swaps two references from inside a method – only the method's local copies swap.

final, abstract, sealed and Packages

Day 7–8. The keywords that *restrict* a hierarchy, modern `switch`, naming rules that graders look for, and how packages control visibility.

final – three uses

Applied to	Effect	Example
Variable	Becomes a constant – exactly one assignment	<code>final int MAX = 10;</code>
Method	Cannot be overridden in a subclass	<code>public final void audit()</code>
Class	Cannot be extended at all	<code>final class String</code>

- A `final` **field** may be initialized in exactly **one** of: declaration, init block, or **every** constructor (a *blank final*). The compiler verifies it is assigned exactly once on every path.
- `final` on a **reference** makes the *reference* constant, **not the object**: `final List<String> l = new ArrayList<>(); l.add("x");` is legal; `l = new ArrayList<>();` is not.

- **final parameters** and locals are also allowed (needed for use inside anonymous classes/lambda before Java 8's "effectively final").

```
class Config {
    final int port;                // blank final - must be set in every
    constructor                    // set at declaration
    final String host = "localhost"; // class constant: static + final + ALL_CAPS
    static final double TAX_RATE = 0.18;

    Config(int port) { this.port = port; } // OK
    // Config() { }                       // compile error: port not initialized
}
```

abstract

- An **abstract method** has a signature but **no body** (`abstract void draw();`). It is a promise that *some* subclass will implement it.
- A class containing **at least one** abstract method **must** be declared **abstract** . (The reverse is not required – an abstract class may have zero abstract methods, purely to prevent instantiation.)
- An abstract class **cannot be instantiated** directly (`new Shape()` is a compile error), but it **can have constructors** (called via `super( ... )`), fields, concrete methods and **final** methods.
- A concrete subclass must implement **all** inherited abstract methods, or itself be abstract.
- **abstract + final on the same class/method is a contradiction** → compile error (abstract says "must be extended/overridden", final says "cannot be").

```
abstract class Shape {
    private final String name;

    Shape(String name) { this.name = name; } // constructors allowed

    abstract double area(); // no body - subclasses decide

    public final String describe() { // cannot be overridden
        return name + " with area " + area(); // template method calls the
    } // abstract one
}

class Circle extends Shape {
    private final double r;
    Circle(double r) { super("Circle"); this.r = r; }
    @Override double area() { return Math.PI * r * r; }
}

Shape s = new Circle(2); // abstract ref → concrete object
System.out.println(s.describe()); // Circle with area 12.566...
```

Sealed classes (Java 17)

- Problem `final` does not solve: "I want *these specific* subclasses and **no others**." `sealed` + `permits` lists exactly who may extend.
- **Every permitted subclass must choose one** of:
 - `final` – the chain ends here.
 - `sealed` – continues with its *own* `permits` list.
 - `non-sealed` – re-opens the hierarchy; anyone may extend it.
- Permitted classes must be in the same module (or same package if unnamed module).

```
sealed class Employee permits Manager, Engineer { }

final class Manager extends Employee { }           // chain ends

non-sealed class Engineer extends Employee { }      // open again – anyone may extend
Engineer

class Intern extends Engineer { }                  // OK (Engineer is non-sealed)
// class Contractor extends Employee { }           // compile error – not in permits list
```

- Benefit: the compiler knows the *complete* set of subtypes, which enables exhaustive `switch` without `default` (Java 21 pattern matching).

Switch expressions (Java 14, standard in 17)

```
// Classic statement – fall-through unless you break
switch (choice) {
    case 1:
        System.out.println("One");
        break;                               // forget this → falls into case 2
    case 2:
        System.out.println("Two");
        break;
    default:
        System.out.println("Other");
}

// Arrow form as an EXPRESSION – no fall-through, no break, returns a value
int result = switch (choice) {
    case 1 → 11;                               // single expression
    case 2, 3 → 22;                             // multiple labels
    default → {                                 // block needs yield
        System.out.println("Default");
        yield 12;                               // value of the switch
    }
};
```

- `yield` returns a value *from the switch block*; `return` would exit the **whole method** – they are not interchangeable.

- A switch **expression** must be **exhaustive** (cover every case or have `default`).
- Arrow form also works as a *statement* (no assigned value) – still no fall-through.

Naming conventions (mandatory, graded)

Element	Style	Example
Class / interface / enum / record	PascalCase	<code>EmployeeManager</code> , <code>Comparable</code>
Method / variable / parameter	camelCase	<code>getSalary()</code> , <code>totalCount</code>
Constant (<code>static final</code>)	ALL_CAPS_WITH_UNDERSCORES	<code>MAX_RETRIES</code>
Package	lowercase reverse-domain	<code>in.cdac.employee</code>
Generic type parameter	Single capital	<code>T</code> , <code>K</code> , <code>V</code> , <code>E</code>

- **No magic numbers.** Replace `if (choice == 1)` with a named constant:

```
final int ADD_MANAGER = 1;
final int ADD_ENGINEER = 2;
final int EXIT = 0;

if (choice == ADD_MANAGER) { ... }
```

Packages

- `package in.cdac.employee;` must be the **first statement** in the file (before imports).
- The **folder structure must mirror** the package: `in/cdac/employee/Employee.java` .
- Compile with `javac -d . Employee.java` (creates the folders), run with `java in.cdac.employee.Program` (fully qualified).
- `import` styles:
 - `import java.util.ArrayList;` – **preferred**, explicit.
 - `import java.util.*;` – **avoid** wildcards: name clashes (`java.util.Date` vs `java.sql.Date`) and unclear dependencies.
 - `java.lang.*` is **auto-imported** (`String` , `System` , `Math` , `Integer` ...).

Access specifiers

Specifier	Same class	Same package	Different package, subclass	Different package, unrelated
<code>private</code>	Yes	No	No	No
<i>(default / package-private)</i>	Yes	Yes	No	No
<code>protected</code>	Yes	Yes	Yes (via inheritance)	No
<code>public</code>	Yes	Yes	Yes	Yes

- **Top-level classes** can only be `public` or default. `private` / `protected` classes are only possible as *nested* classes.
- `protected` = default + access from subclasses in other packages. It is *not* "subclass only".
- Rule of thumb: fields `private` , methods `public` , helpers `private` , hooks for subclasses `protected` .

Class loading – when does the JVM load a class?

Three triggers:

1. First `new` of the class.
 2. First access to a **static member** (`ClassName.staticVar` or `ClassName.staticMethod()`).
 3. Explicit `Class.forName("in.cdac.Base")` (used by JDBC drivers).
- `Base obj = null;` triggers **nothing** – no class loading, no static block.
 - Loading a subclass loads its **parent first**.

Full initialization sequence:

```
CLASS LOAD (once):    static fields (in order) → static blocks (in order)
EACH new (per object): instance fields → instance init blocks → constructor
```

```

class Base {
    static { System.out.println("1. Base static block"); }
    { System.out.println("3. Base init block"); }
    Base() { System.out.println("4. Base constructor"); }
}

class Derived extends Base {
    static { System.out.println("2. Derived static block"); }
    { System.out.println("5. Derived init block"); }
    Derived() { System.out.println("6. Derived constructor"); }

    public static void main(String[] args) {
        new Derived();
        new Derived();           // static blocks do NOT run again; 3-6 repeat
    }
}

```

Strings, Object Class, Wrappers and GC

Day 8–9. The most-asked interview area: string pool, `==` vs `equals`, the `Object` contract, and how Java frees memory.

Strings

- `char` is **2 bytes (Unicode)**, so `String` can hold any language's characters.
- A `String` is an **object** (`java.lang.String`), yet has literal syntax (`"hello"`) – the only class with that privilege.

The String Pool

```

String s1 = "hello";
String s2 = "hello";
String s3 = new String("hello");

```

String Pool (inside heap)



regular heap: `"hello"` ← `s3` (separate object)

- Literals** with the same content **share one object** in the pool – the JVM interns them at class-load time.
- `new String("hello")` **bypasses** the pool and always creates a fresh heap object (the literal `"hello"` inside the parentheses *is* pooled, so this line can create two objects).
- `s3.intern()` returns the pooled instance.

Immutability

- `String` objects **cannot change** after creation. Every "modifying" method (`toUpperCase` , `concat` , `replace` , `+`) returns a **new** `String` .
- **Why:** pool objects are *shared*. If `s1` could mutate `"hello"` to `"jello"` , `s2` would silently change too. Immutability also makes strings thread-safe and safe as `HashMap` keys.

```
String s = "hello";
s.toUpperCase();           // returns "HELLO" - but result is discarded!
System.out.println(s);    // hello (unchanged)
s = s.toUpperCase();       // must reassign
System.out.println(s);    // HELLO
```

Comparing strings

Operation	Compares	Result for <code>"hello"</code> vs <code>new String("hello")</code>
<code>==</code>	Addresses (same object?)	<code>false</code>
<code>.equals()</code>	Content , case-sensitive	<code>true</code>
<code>.equalsIgnoreCase()</code>	Content, ignoring case	<code>true</code>
<code>.compareTo()</code>	Lexicographic order → negative / 0 / positive	<code>0</code>

```
String a = "hello", b = "hello", c = new String("hello");
System.out.println(a == b);           // true - same pooled object
System.out.println(a == c);           // false - different objects
System.out.println(a.equals(c));      // true - same content
System.out.println("Hello".equalsIgnoreCase(a)); // true
```

Rule: never compare strings with `==` in real code. It "works" for literals by accident and fails for anything read from input, files or built at runtime.

Frequently used String methods

```
String s = " Java Programming ";
s.length();           // 20
s.trim();             // "Java Programming"
s.charAt(2);          // 'J'
s.indexOf("Prog");    // 7
s.substring(2, 6);    // "Java" (end index exclusive)
s.contains("gram");   // true
s.split(" ");         // String[] tokens
s.replace('a', 'o');  // " Jovo Progromming "
String.join("-", "a", "b"); // "a-b"
String.format("%d-%s", 5, "x"); // "5-x"
"a,b,c".split(",").length; // 3
```

Concatenation cost and StringBuffer / StringBuilder

- "hello" + "bye" + 42 inside a loop creates an **intermediate String object on every +** – garbage piles up.
- **StringBuffer** (thread-safe, synchronized) and **StringBuilder** (Java 5, not synchronized, faster) are **mutable** character buffers.

```
StringBuffer buffer = new StringBuffer("hello");
buffer.append("bye").append(42);           // mutates in place, returns 'this' → chaining
buffer.insert(0, ">> ");
buffer.reverse();
String str = buffer.toString();             // convert back when done

// Rule: single-threaded → StringBuilder; shared between threads → StringBuffer
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) sb.append(i).append(',');
System.out.println(sb);                    // 0,1,2,3,4,
```

	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Speed	Slow for repeated concat	Fastest	Slower than builder

The Object class

- Every class **implicitly extends java.lang.Object** – so every object has these methods:

Method	Default behavior	Should you override?
<code>String toString()</code>	<code>ClassName@hexHashCode</code>	Yes – return meaningful state
<code>boolean equals(Object o)</code>	Reference comparison (<code>==</code>)	Yes if objects have logical equality
<code>int hashCode()</code>	Address-derived int	Yes, together with <code>equals</code> (Section 13)
<code>Object clone()</code>	Shallow field copy (needs <code>Cloneable</code>)	Rarely – prefer copy constructors
<code>Class<?> getClass()</code>	Runtime class object (RTCI)	<code>final</code> – cannot
<code>void finalize()</code>	Nothing	Deprecated (Java 9) – never
<code>wait()</code> / <code>notify()</code> / <code>notifyAll()</code>	Thread coordination (Section 15)	<code>final</code> – cannot

```

class Point {
    private final int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public String toString() { return "Point(" + x + ", " + y + ")"; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;           // same reference
        if (!(obj instanceof Point)) return false; // also handles null
        Point other = (Point) obj;
        return x == other.x && y == other.y;
    }

    @Override
    public int hashCode() { return java.util.Objects.hash(x, y); }
}

Point p = new Point(1, 2);
System.out.println(p);           // Point(1, 2) - println calls toString()
System.out.println(p.equals(new Point(1, 2))); // true
System.out.println(p == new Point(1, 2));     // false

```

Boxing and unboxing

- **Boxing:** primitive → wrapper object. **Unboxing:** wrapper → primitive. Since Java 5 both are **automatic**.

```

Integer data = 5;                // auto-boxing (Integer.valueOf(5))
int num = data;                  // auto-unboxing (data.intValue())

List<Integer> list = new ArrayList<>();
list.add(10);                    // boxing - generics need objects
int first = list.get(0);         // unboxing

Integer n = null;
int crash = n;                   // NullPointerException - unboxing null!

```

- `Integer.valueOf` caches -128 to 127, so `Integer a = 127, b = 127; a == b` is `true`, but for 128 it is `false`. Always use `.equals()` for wrappers.

Memory management and garbage collection

- Java has **no delete operator and no destructors**. This removes the two biggest C/C++ bugs at once: **dangling pointers** (freeing too early) and **memory leaks** (never freeing).
- The JVM **pre-allocates a memory pool** (the *managed heap*) at startup and **grows/shrinks it adaptively** (`-Xms` , `-Xmx` set the bounds).

Reference counting model (conceptual)

Reference count increases when	Reference count decreases when
<code>new</code> + assignment: <code>Data d = new Data();</code>	Set to null: <code>d = null;</code>
Reference-to-reference copy: <code>Data e = d;</code>	Reassignment: <code>d = new Data();</code> (old object loses one)
Object passed to / returned from a method	Reference goes out of scope (method returns)

```

Data d1 = new Data();           // obj A: RC = 1
Data d2 = d1;                   // obj A: RC = 2
d1 = new Data();                // obj A: RC = 1 (d1 now → obj B, RC = 1)
d2 = null;                      // obj A: RC = 0 → eligible for GC

```

- Real HotSpot collectors use **reachability tracing** (mark-and-sweep from GC roots) rather than pure counting – it handles cycles – but the *eligibility rule* is the same: an object nobody can reach is garbage.

The garbage collector

- A **low-priority background JVM thread** (daemon). It destroys unreachable objects and compacts memory when the JVM decides – usually when the heap is under pressure.
- `System.gc()` only *requests* a collection (raises GC priority) – **no guarantee** it runs. Never rely on it.
- **G1 (Garbage-First)** is the default collector since Java 9. Others: Serial, Parallel, ZGC, Shenandoah.
- `finalize()` is **deprecated** – its timing was unpredictable (may never run). Release files, sockets and DB connections **explicitly** with `close()` or *try-with-resources* (Section 9).

Exception Handling

Day 9–10. The vocabulary, the five keywords, the hierarchy, and how to design your own exceptions.

Definitions

- **Exception:** an *unpredictable runtime situation* that causes **abrupt termination** of the normal flow (file missing, network down, divide by zero).
- **Exception handling:** defining the **corrective action** so the program recovers or fails gracefully instead of crashing.
- Compile errors are *not* exceptions – exceptions happen while the program **runs**.

The five keywords

Keyword	Role
<code>try</code>	Wraps the code that might fail
<code>catch</code>	Handles a specific exception type
<code>finally</code>	Always runs – cleanup
<code>throw</code>	Raise an exception object explicitly
<code>throws</code>	Declare that a method may propagate an exception to its caller

```
import java.io.*;
import java.sql.SQLException;

public void process() {
    try {
        connectToDatabase();           // may throw SQLException
        writeToFile();                 // may throw IOException /
        FileNotFoundException
    } catch (FileNotFoundException e) { // DERIVED-most FIRST
        System.out.println("Invalid file path: " + e.getMessage());
    } catch (IOException e) {          // BASE-most last
        System.out.println("IO error.");
    } catch (SQLException | InterruptedException e) { // multi-catch: unrelated types only
        System.out.println("Other error.");
    } finally {
        // ALWAYS runs – whether try succeeded, a catch ran, or an exception escaped.
        // Only System.exit() or a JVM crash skips it. Release resources here.
        System.out.println("Cleanup");
    }
}
```

Rules worth memorizing

- **Catch order = derived first, base last.** If `IOException` is caught before `FileNotFoundException`, the second catch is **unreachable** → **compile error**.
- **Multi-catch** (`A | B`) types must **not** be related by inheritance (compile error otherwise); the variable `e` is implicitly `final`.
- **finally runs even after return** inside `try`. If `finally` itself returns, it *overrides* the try's return value (bad practice).
- `try` needs **at least one** `catch` **or** a `finally`.
- Only **Throwable subclasses** can be thrown – `throw 42;` or `throw "error";` are compile errors (unlike C++, which can throw anything).

throw and throws

```
// throw - raise NOW, inside the method
public void withdraw(double amount) {
    if (amount > balance)
        throw new IllegalArgumentException("Insufficient balance"); // unchecked
    balance -= amount;
}

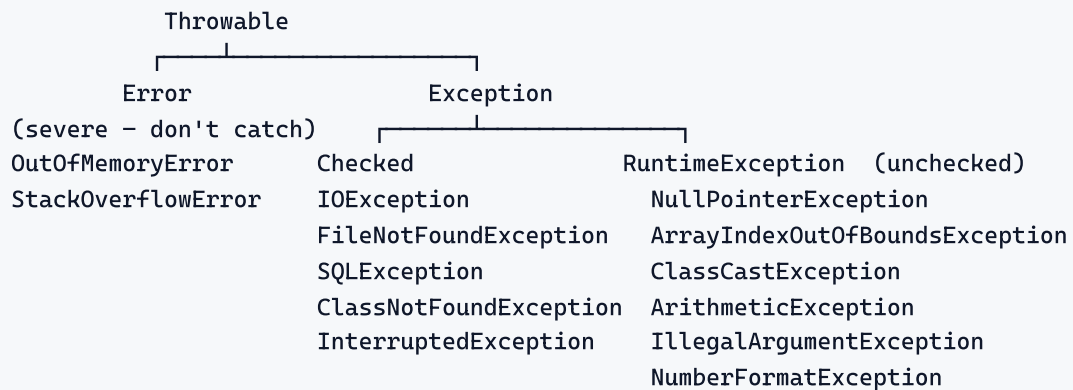
// throws - "I don't handle it; caller must"
public void readConfig() throws IOException { // mandatory for CHECKED exceptions
    FileInputStream fis = new FileInputStream("config.txt");
}

// Caller has two choices:
public void caller() throws IOException { // 1. propagate further
    readConfig();
}

public void caller2() { // 2. handle it
    try { readConfig(); } catch (IOException e) { System.out.println("no config"); }
}
```

- If a checked exception is neither caught nor declared: **"unreported exception; must be caught or declared to be thrown"** compile error.
- `main` may declare `throws Exception` – then an unhandled exception terminates the program with a stack trace.

The hierarchy



	Checked	Unchecked
Extends	Exception (but not RuntimeException)	RuntimeException
Compiler forces handling?	Yes – catch or throws	No
Represents	External problems you can <i>anticipate</i> (I/O, DB, network)	Programmer negligence – bugs you should fix, not catch
Examples	IOException , SQLException , FileNotFoundException , InterruptedException	NullPointerException , ArrayIndexOutOfBoundsException , ClassCastException , ArithmeticException

- **Error** subclasses (`OutOfMemoryError` , `StackOverflowError`) signal JVM-level failure – **do not catch** them; the program cannot recover meaningfully.

Custom exceptions

```
// Checked – caller MUST handle. Extend RuntimeException instead for unchecked.
class DateException extends Exception {
    public DateException(String message) {
        super(message); // Throwable stores it → getMessage()
    }
    public DateException(String message, Throwable cause) {
        super(message, cause); // chaining – keep the original
    }
}

public class Date {
    public void setDate(int day, int month, int year) throws DateException {
        if (month < 1 || month > 12) throw new DateException("Invalid value of month: " +
month);
        if (day < 1 || day > 31) throw new DateException("Invalid value of day: " +
day);
        // ...
    }
}
```

- Naming: always end with `Exception`. Provide at least a `(String)` constructor; add `(String, Throwable)` for chaining.
- Prefer **unchecked** for validation of programmer input (`IllegalArgumentException` style); **checked** when the caller can realistically recover.

Rethrowing and chaining

```
try {
    repository.save(order);
} catch (SQLException e) {
    log(e);
    throw e; // 1. rethrow as-is
}

try {
    repository.save(order);
} catch (SQLException e) {
    throw new OrderException("Could not save order " + order.id(), e); // 2. wrap with
context
}
// Later: caught.getCause() gives the original SQLException; stack trace shows "Caused by:"
```

- `finally` runs **before** a rethrown exception leaves the method.
- Useful `Throwable` methods: `getMessage()`, `getCause()`, `printStackTrace()`, `getStackTrace()`.

Nested try-catch – protect the outer loop

```
Scanner sc = new Scanner(System.in);
int choice;
do {
    choice = sc.nextInt();
    try {                                     // inner try isolates ONE operation
        switch (choice) {
            case 1 → date.setDate(sc.nextInt(), sc.nextInt(), sc.nextInt());
            case 2 → date.addDays(sc.nextInt());
        }
    } catch (DateException e) {
        System.out.println("Error: " + e.getMessage()); // menu loop continues
    }
} while (choice ≠ 0);
```

- Without the inner `try`, one bad date would propagate out and **kill the whole menu loop**. Isolating failures keeps the application alive.

File I/O and Serialization

Day 10–11. `java.io` from raw bytes up to whole objects, plus the resource-management idiom you should always use.

File – metadata only

`java.io.File` represents a **path** (file or directory). It can create, delete and inspect – it **cannot read or write content**.

```
import java.io.File;

File f = new File("E:\\DAC\\test.txt");    // escape backslashes, or use "E:/DAC/test.txt"

f.exists();           // does it exist?
f.createNewFile();    // create empty file (throws IOException) - returns false if exists
f.isFile();           // regular file?
f.isDirectory();      // directory?
f.length();           // size in bytes
f.getName();          // "test.txt"
f.getAbsolutePath(); // full path
f.delete();           // PERMANENT - no recycle bin!

File dir = new File("E:\\DAC\\reports\\2026");
dir.mkdir();          // creates only the LAST folder (fails if parents missing)
dir.mkdirs();         // creates the whole chain
String[] names = dir.list();           // names of entries
File[] entries = dir.listFiles();      // File objects
```

Stream hierarchy overview

BYTE streams (binary, 8-bit)	CHARACTER streams (text, 16-bit)
InputStream / OutputStream	Reader / Writer
├ FileInputStream / FileOutputStream	├ FileReader / FileWriter
├ DataInputStream / DataOutputStream	├ BufferedReader / BufferedWriter
├ ObjectInputStream / ObjectOutputStream	└ InputStreamReader (bridge bytes→chars)
└ BufferedInputStream / BufferedOutputStream	

- **Decorator pattern:** `new DataOutputStream(new FileOutputStream("x.dat"))` – wrap a low-level stream to add capabilities.

Byte streams: `FileOutputStream` / `FileInputStream`

```
// WRITE
FileOutputStream fos = new FileOutputStream("test.txt");           // creates / truncates
// new FileOutputStream("test.txt", true) → append mode
fos.write("Welcome to Java!".getBytes());                        // String → byte[]
fos.close();

// READ
File main = new File("test.txt");
FileInputStream fis = new FileInputStream(main);
byte[] arr = new byte[(int) main.length()];                      // exact size buffer
fis.read(arr);                                                    // fills the array
System.out.println(new String(arr));                              // byte[] → String
fis.close();

// Java 9+ shortcut
byte[] all = new FileInputStream("test.txt").readAllBytes();
```

- `read()` with no args returns **one byte as `int`** (0–255) or **–1 at end of file** – the loop idiom:

```
int b;
while ((b = fis.read()) != -1) {
    System.out.print((char) b);
}
```

Primitive I/O: `DataOutputStream` / `DataInputStream`

- Write primitives and strings in **binary form** (an `int` takes exactly 4 bytes, not its decimal text).
- **The read order must exactly match the write order** – there is no metadata in the file to tell you what comes next.

```
// WRITE one record
try (DataOutputStream ds = new DataOutputStream(new FileOutputStream("emp.dat"))) {
    ds.writeUTF("Manager");           // String (length-prefixed)
    ds.writeFloat(75000.5f);
    ds.writeInt(35);
    ds.writeBoolean(true);
}

// READ many records until EOF
try (DataInputStream di = new DataInputStream(new FileInputStream("emp.dat"))) {
    while (true) {
        String role   = di.readUTF();           // same order: UTF → float → int → boolean
        float salary  = di.readFloat();
        int age       = di.readInt();
        boolean perm  = di.readBoolean();
        System.out.println(role + " " + salary + " " + age + " " + perm);
    }
} catch (EOFException eof) {
    // EXPECTED – this is how DataInputStream signals end of file
    System.out.println("All records read.");
}
```

Serialization: writing whole objects

- **Persistence** = saving data only. **Serialization** = converting an object to a byte stream **with metadata** (class name, field names/types), so it can be reconstructed – *deserialized* – later or on another machine.
- `java.io.Serializable` is a **marker interface** – it has **no methods**. It merely *tags* the class as "allowed to be serialized". Without it → `NotSerializableException` at runtime.
- **transient** fields are **skipped** during serialization and get **default values** (`0`, `null`, `false`) on read – use for passwords, caches, derived fields, non-serializable members.
- **static** fields are also not serialized (they belong to the class, not the object).
- All **referenced objects** must be `Serializable` too (deep graph is written).

```

import java.io.*;

public class Student implements Serializable {
    private static final long serialVersionUID = 1L;    // version stamp - avoids
    InvalidClassException after edits

    private String name;
    private transient int age;                        // NOT written to the file
    private transient String password;                // sensitive - never persisted

    public Student(String name, int age, String password) {
        this.name = name; this.age = age; this.password = password;
    }

    @Override
    public String toString() { return name + " (" + age + ") pw=" + password; }
}

class SerializeDemo {
    public static void main(String[] args) throws IOException, ClassNotFoundException {
        // WRITE
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("students.dat"))) {
            oos.writeObject(new Student("Shivam", 22, "secret"));
            oos.writeObject(new Student("Asha", 23, "hidden"));
        }

        // READ - downcast required because readObject() returns Object
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("students.dat"))) {
            while (true) {
                Student s = (Student) ois.readObject();
                System.out.println(s);    // Shivam (0) pw=null - transient fields reset
            }
        } catch (EOFException eof) {
            System.out.println("Done");
        }
    }
}

```

Polymorphic collections and the `ClassCastException` bug

```

// WRONG: file contains both Managers and Engineers
Manager[] staff = new Manager[10];
staff[i] = (Manager) ois.readObject();    // ClassCastException when an Engineer appears

// RIGHT: type the array/list as the BASE
Employee[] staff = new Employee[10];
staff[i] = (Employee) ois.readObject();    // every subtype IS-A Employee - always safe

```

try-with-resources (Java 7)

- Any class implementing `AutoCloseable` (or its sub-interface `Closeable`) can be declared in the `try ( ... )` header; its `close()` is called **automatically**, even if an exception occurs.
- Multiple resources are closed in **reverse order** of declaration (last opened, first closed).
- Eliminates the verbose `finally { if (fos != null) fos.close(); }` pattern and the risk of forgetting it.

```
try (FileOutputStream fos = new FileOutputStream("s.dat");
    ObjectOutputStream oos = new ObjectOutputStream(fos)) {    // oos closed first, then
    fos                                                         fos
    oos.writeObject(student);
} catch (IOException e) {
    e.printStackTrace();
}
// no finally needed

// Your own resource
class Connection implements AutoCloseable {
    @Override public void close() { System.out.println("Connection closed"); }
}
try (Connection c = new Connection()) { /* use it */ }    // prints "Connection closed"
```

Interfaces

Day 10, 13. The contract mechanism that gives Java multiple inheritance of *type*, and the Java 8/9 features that changed what an interface can contain.

What an interface is

- An interface = a **pure abstract type**: a set of method **signatures** (the contract) with no state.
- Methods are implicitly `public abstract`; fields are implicitly `public static final` (constants).
- A class **implements** an interface and must provide bodies for **all** abstract methods (or be abstract itself).
- A class can implement **many** interfaces; an interface can **extends** many interfaces.
- You **cannot instantiate** an interface, but you can hold a reference of its type – that reference sees **only the interface's methods**.

```

interface Shape {
    double PI = 3.14159;           // public static final
    double area();                 // public abstract
    double perimeter();
}

class Circle implements Shape {
    private final double r;
    Circle(double r) { this.r = r; }
    @Override public double area()    { return PI * r * r; }    // MUST be public
    @Override public double perimeter() { return 2 * PI * r; }
}

Shape s = new Circle(1);           // interface reference → concrete object
System.out.println(s.area());

```

Java 8+ additions

Member	Keyword	Body?	Inherited by implementer?	Overridable?	Since
Abstract method	<i>(none)</i>	No	Yes (must implement)	Must	1.0
Default method	default	Yes	Yes	Yes (optional)	8
Static method	static	Yes	No – call as <code>Interface.method()</code>	No	8
Private method	private	Yes	No – helper for default/static methods	No	9

```

interface Operation {
    void operate();                // abstract - implementer must
    define

    default void show() {          // inherited; may be
    overridden
        System.out.println("default show → " + describe());
    }

    static void init() {           // Operation.init() only
        System.out.println("static init");
    }

    private String describe() {    // Java 9+ - shared helper,
    hidden
        return "Operation helper";
    }
}

class Test implements Operation {
    @Override public void operate() { System.out.println("operating"); }
}

Operation op = new Test();
op.operate();           // operating
op.show();              // default show → Operation helper
Operation.init();       // static init (op.init() is a compile error)

```

Why default methods exist. Java 8 wanted to add `stream()` to the `Collection` interface. Without defaults, every existing implementation in the world (`ArrayList` , `HashSet` , thousands of user classes) would have failed to compile. A *default* method gives them an inherited body for free.

Multiple inheritance via interfaces

```

interface Flyer { default void move() { System.out.println("fly"); } }
interface Swimmer { default void move() { System.out.println("swim"); } }

class Duck implements Flyer, Swimmer {
    @Override
    public void move() {           // MUST override - compiler forces resolution
        Flyer.super.move();       // pick explicitly
        Swimmer.super.move();
    }
}

```

- **Pre-Java 8** there was no diamond problem at all: interface methods had no bodies, so inheriting two identical abstract signatures is harmless – the implementer writes the single body.
- **Java 8+:** if two defaults clash, the class **must override** and may call `Interface.super.method()` – ambiguity is resolved *explicitly*, never silently.

Interface as a contract and access control

Pass an **interface-typed reference** to expose only a **subset** of an object's methods.

```
interface Profile {                                // the public "view"
    String getName();
    void setName(String name);
}

class User implements Profile {
    private String name;
    private String email;                          // internal

    public String getName() { return name; }
    public void setName(String n) { this.name = n; }
    public String getEmail() { return email; }      // NOT in Profile
}

class ProfileEditor {
    void changeProfile(Profile p) {                // receives the interface, not User
        p.setName("New Name");
        // p.getEmail();                          // compile error - hidden behind the contract
    }
}
```

- The same object, seen through a narrower interface, is a form of **hiding without private** – the caller *cannot* reach what the interface does not declare (without a downcast).

Sealed interfaces (Java 17)

```
sealed interface Operation permits Add, Subtract { }
final class Add implements Operation { }
final class Subtract implements Operation { }
// class Multiply implements Operation { } → compile error
```

Overriding rule: visibility may widen, never narrow

- Interface methods are `public`, so the implementing method **must be public**. Writing `void area()` (package-private) in `Circle` is a compile error: *"attempting to assign weaker access privileges"*.
- In class inheritance: `protected` → `public` is fine; `public` → `protected` is not. Reason: a base-typed reference must always be able to call what the base promised.

Abstract class vs interface

	Abstract class	Interface
Inheritance	Single (<code>extends</code>)	Multiple (<code>implements</code>)
State	Instance fields allowed	Only <code>public static final</code> constants
Constructors	Yes	No
Method types	Abstract, concrete, final, static, any access	Abstract, default, static, private
Use when	Sharing code + state among closely related classes (<i>is-a</i> with common base)	Defining a capability/contract across unrelated classes (<i>can-do</i> : <code>Comparable</code> , <code>Runnable</code>)

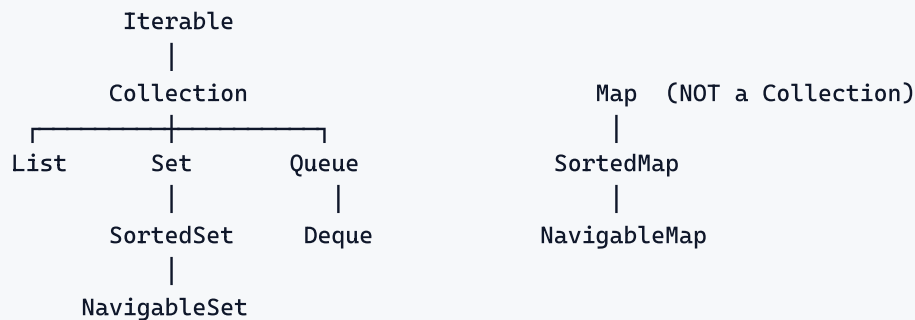
Collections Framework

Day 11–12. Data structures done right: the interface hierarchy, when to use which implementation, cursors, generics, and the two ways to sort.

Why collections – array drawbacks

Array problem	Collection fix
Fixed size – decided at creation, cannot grow	Grows/shrinks automatically
Predefined size ⇒ memory wastage or overflow	Allocates as needed
No underlying data structure (just contiguous memory)	Linked list, hash table, tree... chosen per class
No supporting methods (search, sort, remove)	Rich API: <code>add</code> , <code>remove</code> , <code>contains</code> , <code>sort</code> ...
Holds primitives or objects	Objects only (wrappers via auto-boxing)

The 9 core interfaces



- **Collection, List, Set, SortedSet, NavigableSet, Queue, Map, SortedMap, NavigableMap.**
- **Map** does not inherit **Collection** – it stores *pairs*, not elements.
- **No concrete class implements Collection directly** – you always use **List / Set / Queue** implementations.
- **Collection (interface) ≠ Collections (utility class with static helpers)** – classic interview trap.

Interface	Duplicates	Order	Key implementations
List	Yes	Insertion, index-based	ArrayList , LinkedList , Vector , Stack
Set	No	Unordered / insertion / sorted	HashSet , LinkedHashSet , TreeSet
Queue / Deque	Yes	FIFO / both ends	LinkedList , ArrayDeque , PriorityQueue
Map	Keys no, values yes	Unordered / insertion / sorted by key	HashMap , LinkedHashMap , TreeMap , Hashtable

Cursors – three ways to iterate

Cursor	Direction	Applies to	Operations	Since
Enumeration (legacy)	Forward	Vector , Stack , Hashtable	read (hasMoreElements , nextElement)	1.0
Iterator (universal)	Forward	All collections	read, remove	1.2
ListIterator	Bidirectional	List only	read, remove, add , set	1.2

```

List<String> list = new ArrayList<>(List.of("A", "B", "C"));

// Iterator - ALWAYS guard with hasNext() or you get NoSuchElementException
Iterator<String> itr = list.iterator();
while (itr.hasNext()) {
    String s = itr.next();
    if (s.equals("B")) itr.remove();           // safe removal during iteration
}
// list.remove(...) inside a for-each → ConcurrentModificationException

// ListIterator - both directions, modification
ListIterator<String> li = list.listIterator();
while (li.hasNext()) { String s = li.next(); li.set(s.toLowerCase()); } // forward
while (li.hasPrevious()) { System.out.println(li.previous()); }        // backward

// Enumeration - legacy
Vector<Integer> v = new Vector<>(List.of(1, 2, 3));
Enumeration<Integer> en = v.elements();
while (en.hasMoreElements()) System.out.println(en.nextElement());

```

- Internally every collection implements **Iterable** and its `iterator()` returns an instance of a **private inner class** implementing **Iterator** that knows the collection's internals. The enhanced **for** loop is just sugar over `iterator()` / `hasNext()` / `next()`.

List implementations

All lists: **duplicates allowed, insertion order preserved, nulls allowed, index access.**

	ArrayList	LinkedList	Vector
Structure	Growable array (grows by ~50%)	Doubly linked list	Growable array (grows by 100%)
RandomAccess	Yes – <code>get(i)</code> is O(1)	No – <code>get(i)</code> is O(n)	Yes
Insert/delete in middle	Costly – shifts elements (array copy)	O(1) once positioned	Costly
Thread-safe	No	No	Yes – all methods synchronized (slower)
Best for	Retrieval-heavy work	Insert/delete-heavy work	Legacy multithreaded code

```

List<Integer> arr = new ArrayList<>();
arr.add(10); arr.add(20); arr.add(0, 5);           // [5, 10, 20]
arr.get(1);                                       // 10
arr.set(1, 15);                                  // [5, 15, 20]
arr.remove(Integer.valueOf(20));                  // remove by OBJECT → [5, 15]
arr.remove(0);                                   // remove by INDEX → [15]
arr.contains(15); arr.indexOf(15); arr.size(); arr.isEmpty();

LinkedList<String> ll = new LinkedList<>();
ll.addFirst("a"); ll.addLast("z"); ll.removeFirst(); // also a Deque

Stack<Integer> st = new Stack<>();                // extends Vector - LIFO
st.push(1); st.push(2); st.push(3);
st.peek(); // 3 (look, don't remove)
st.pop(); // 3 (remove)
st.search(1); // 1-based position from top → 2

```

Generics

- Generics give **compile-time type safety** – the compiler rejects wrong types and inserts casts for you.

```

// Raw (pre-Java 5): anything goes, casts everywhere, runtime ClassCastException risk
List raw = new ArrayList();
raw.add("text"); raw.add(42);
String s = (String) raw.get(1); // compiles, explodes at runtime

// Generic
List<String> list = new ArrayList<>(); // diamond <> infers type (Java 7+)
list.add("text");
// list.add(42); // compile error - caught early
String first = list.get(0); // no downcast needed

```

```

// Your own generic class with multiple type parameters (like Map<K,V>)
class Pair<T, U> {
    private final T first;
    private final U second;
    Pair(T first, U second) { this.first = first; this.second = second; }
    T getFirst() { return first; }
    U getSecond() { return second; }
}
Pair<String, Integer> p = new Pair<>("age", 30);

// Generic method - type parameter declared before the return type
static <T> void printAll(List<T> items) { for (T t : items) System.out.println(t); }

```

- No primitives** as type arguments: `List<int>` is illegal → use `List<Integer>` (auto-boxing hides the conversion). This differs from C++ templates, which accept `vector<int>`.
- Type erasure**: generics exist only at compile time; at runtime `List<String>` and `List<Integer>` are the same `List` class. Hence you cannot do `new T()` or `T[]`.

Collections utility class

```
List<Integer> nums = new ArrayList<>(List.of(40, 90, 78, 12));

Collections.sort(nums);           // [12, 40, 78, 90] - natural order
Collections.reverse(nums);        // [90, 78, 40, 12]
Collections.max(nums);            // 90
Collections.min(nums);           // 12
Collections.shuffle(nums);
Collections.frequency(nums, 40);  // 1
Collections.sort(nums);
Collections.binarySearch(nums, 78); // index 2 - list MUST be sorted first
Collections.unmodifiableList(nums); // read-only view
Collections.synchronizedList(nums); // thread-safe wrapper
```

Arrays.asList and varargs

```
List<Integer> fixed = Arrays.asList(40, 90, 78); // fixed-size view backed by the array
fixed.set(0, 1);                               // OK
// fixed.add(5);                               // UnsupportedOperationException

List<Integer> growable = new ArrayList<>(Arrays.asList(40, 90, 78)); // copy → growable
List<Integer> immutable = List.of(1, 2, 3); // Java 9+ - fully immutable
```

- `T ... values` = **varargs**: the compiler packs the arguments into an array. Must be the **last** parameter; only one per method.

```
public static int sum(int ... data) { // one method, any argument count
    int total = 0;
    for (int d : data) total += d;
    return total;
}

sum(); // 0
sum(1, 2, 3); // 6
sum(new int[]{4, 5}); // also accepted - it IS an array
```

Sorting objects: Comparable vs Comparator

	Comparable<T>	Comparator<T>
Package	java.lang	java.util
Method	int compareTo(T other) – one argument	int compare(T a, T b) – two arguments
Where	Inside the class being sorted	Separate class (or lambda)
Orders	One natural order	Many alternative orders
Used by	Collections.sort(list), TreeSet, TreeMap	Collections.sort(list, cmp), list.sort(cmp), new TreeSet<>(cmp)
Return contract	negative if this < other, 0 if equal, positive if this > other	same, for a vs b

```
class Employee implements Comparable<Employee> {
    String name; int age; double salary;
    Employee(String n, int a, double s) { name = n; age = a; salary = s; }

    @Override
    public int compareTo(Employee other) {           // natural order: by age
        return Integer.compare(this.age, other.age); // safer than subtraction (no
    } // overflow)
    @Override public String toString() { return name + "/" + age + "/" + salary; }
}

// Comparator - alternative order, kept OUTSIDE the class
class EmployeeSalaryComparator implements Comparator<Employee> {
    @Override
    public int compare(Employee o1, Employee o2) {
        return Double.compare(o1.salary, o2.salary);
    }
}

List<Employee> list = new ArrayList<>(List.of(
    new Employee("Asha", 30, 90000), new Employee("Ravi", 25, 60000), new Employee("Meera",
    35, 75000)));

Collections.sort(list); // Comparable → by age
Collections.sort(list, new EmployeeSalaryComparator()); // Comparator → by salary
list.sort(Comparator.comparing(e → e.name)); // lambda-based (Section 12)
list.sort(Comparator.comparingDouble((Employee e) → e.salary).reversed());
```

Set implementations

Common rules: **no duplicates** (add() returns false silently, no exception), **at most one null**, **no index access**, **no ListIterator**.

	HashSet	LinkedHashSet	TreeSet
Structure	Hash table (backed by a <code>HashMap</code>)	Hash table + linked list	Red-black binary search tree
Order	None (bucket order)	Insertion order	Sorted (natural / comparator)
Speed	Fastest – O(1) add/contains	Slightly slower	O(log n)
<code>null</code>	One allowed	One allowed	Not allowed (<code>NullPointerException</code>)
Needs	<code>hashCode()</code> + <code>equals()</code>	same	<code>Comparable</code> or a <code>Comparator</code>

```

Set<String> hs = new HashSet<>();
hs.add("banana"); hs.add("apple"); hs.add("apple"); // second apple → false
System.out.println(hs);                          // order unpredictable

Set<String> ts = new TreeSet<>();
ts.addAll(List.of("banana", "apple", "cherry"));
System.out.println(ts);                          // [apple, banana, cherry]

// TreeSet with custom objects
TreeSet<Employee> byAge = new TreeSet<>(list); // uses compareTo
TreeSet<Employee> bySalary = new TreeSet<>(new EmployeeSalaryComparator());
// new TreeSet<>().add(new Object()) → ClassCastException (not Comparable)

```

- **How `TreeSet` inserts:** compare with root → lesser goes **left**, greater goes **right**, repeat until a free slot. Therefore elements **must be comparable** or you must **pass a `Comparator` to the constructor**.
- `compareTo() == 0` ⇒ **treated as a duplicate** and *not* inserted – even if `equals()` says they differ. Make comparators consistent with equals.

Map implementations

- Stores **key** → **value** pairs. **Duplicate keys overwrite** the old value (`put` returns the previous value); **duplicate values are fine**; one `null` key (`HashMap`), many `null` values.

	HashMap	LinkedHashMap	TreeMap	Hashtable
Order	None	Insertion	Sorted by key	None
<code>null</code> key	One	One	No	No
Thread-safe	No	No	No	Yes (legacy, slow)
Speed	O(1)	O(1)	O(log n)	O(1)

```

Map<Integer, String> map = new HashMap<>();
map.put(1001, "Asha");
map.put(1002, "Ravi");
map.put(1001, "Meera"); // overwrites Asha → returns "Asha"

// Four ways to read
String v = map.get(1002); // 1. by key (null if absent)
Collection<String> vals = map.values(); // 2. all values
Set<Integer> keys = map.keySet(); // 3. all keys
for (Map.Entry<Integer, String> e : map.entrySet()) // 4. all pairs
    System.out.println(e.getKey() + " → " + e.getValue());

map.containsKey(1001); map.containsValue("Ravi"); map.remove(1002); map.size();
map.getOrDefault(9999, "Unknown"); // avoid null checks
map.putIfAbsent(1003, "New");
map.forEach((k, val) → System.out.println(k + "=" + val)); // Java 8

// Word-count idiom
Map<String, Integer> freq = new TreeMap<>(); // sorted keys
for (String w : "a b a c b a".split(" "))
    freq.merge(w, 1, Integer::sum); // {a=3, b=2, c=1}

```

- **TreeMap** sorts by key – custom key objects need `Comparable` or a `Comparator` in the constructor (same rule as `TreeSet`).
- **HashMap internals:** `hashCode()` of the key → bucket index; `equals()` resolves collisions inside the bucket. Wrong/missing overrides ⇒ lost entries (Section 13).

Arrays: matrix vs jagged

```

int[][] matrix = new int[3][4]; // 3 rows × 4 cols – all rows equal length
matrix[1][2] = 7;

int[][] jag = new int[3][]; // 3 rows, column count decided per row
jag[0] = new int[2];
jag[1] = new int[5];
jag[2] = new int[1];
// jag[0][3] = 1; // ArrayIndexOutOfBoundsException – row 0 has 2 slots

int[][] literal = {{1, 2}, {3, 4, 5}, {6}}; // jagged literal
for (int[] row : literal) // row-by-row traversal
    System.out.println(Arrays.toString(row));
System.out.println(literal.length); // 3 rows
System.out.println(literal[1].length); // 3 columns in row 1

```

- A 2-D array in Java is really an **array of array references** – each row is its own heap object, which is why rows can differ in length.

Functional Interfaces and Lambdas

Day 13–14. How Java 8 made behavior a first-class value, and the small syntax that unlocks the whole Stream API.

Functional interface

- An interface with **exactly one abstract method** (counted across its **whole hierarchy**, and excluding `Object`'s public methods such as `equals`).
- It may have **any number** of `default`, `static` and `private` methods.
- `@FunctionalInterface` is **optional** – it is a *compiler guard* that fails the build if someone adds a second abstract method. It is *not* what makes the interface functional.
- Familiar examples: `Runnable` (`run`), `Comparable` (`compareTo`), `Comparator` (`compare`), `ActionListener`.

```
@FunctionalInterface
interface PasswordValidator {
    boolean validate(String password);           // the single abstract method
    default void log() { System.out.println("validating"); } // allowed
}
```

Lambda syntax progression

A **lambda** is an *anonymous function*: `(parameters) → { body }`. It can only be assigned to a **functional interface** – the compiler maps it onto the one abstract method.

```
// 1. Full form - types, parentheses, braces, return
PasswordValidator pv1 = (String str) → {
    if (str.length() ≥ 5) return true;
    else return false;
};

// 2. Drop the parameter type - compiler infers it from validate(String)
PasswordValidator pv2 = (str) → { return str.length() ≥ 5; };

// 3. Single parameter → parentheses optional
PasswordValidator pv3 = str → { return str.length() ≥ 5; };

// 4. Single expression → drop braces and 'return'
PasswordValidator pv4 = str → str.length() ≥ 5;

System.out.println(pv4.validate("abc"));    // false
System.out.println(pv4.validate("abcdef")); // true

// Zero params: () → ...           Two or more: parentheses MANDATORY: (a, b) → a + b
```

- A lambda can read local variables from the enclosing scope only if they are **effectively final** (never reassigned).

Lambda vs anonymous class – the 7 differences

#	Aspect	Anonymous class	Lambda
1	Nature	A class without a name	A function without a name
2	Binds to	Abstract class, concrete class or any interface	Functional interface only
3	.class file	Generated (Program\$1.class)	Not generated (uses invokedynamic)
4	this keyword	Refers to the anonymous object	Refers to the enclosing class instance
5	Instance variables / state	Allowed	Not allowed (no fields)
6	Constructor / init block	Possible (init block)	None
7	Can override multiple methods	Yes	Only the single abstract method

```
// Anonymous class - verbose, but can hold state and bind to a class
Runnable r1 = new Runnable() {
    int count = 0; // allowed
    @Override public void run() { System.out.println("run " + (++count) + " " + this); } //
    this = anon object
};

// Lambda - concise, stateless
Runnable r2 = () → System.out.println("run");
```

java.util.function – the core four

Interface	Abstract method	Takes → Returns	Typical use
Predicate<T>	boolean test(T t)	T → boolean	filtering, validation
Function<T,R>	R apply(T t)	T → R	transformation / mapping
Supplier<T>	T get()	() → T	lazy creation, factories
Consumer<T>	void accept(T t)	T → void	side effects (printing, saving)

```
import java.util.function.*;

Predicate<Integer> isEven = n → n % 2 == 0;
Function<String, Integer> len = str → str.length();
Supplier<String> greet = () → "Hello!";
Consumer<String> print = str → System.out.println(str);

isEven.test(4);           // true
len.apply("Java");        // 4
greet.get();              // "Hello!"
print.accept("Hi");       // prints Hi

// Chaining
Predicate<Integer> isPositive = n → n > 0;
isEven.and(isPositive).test(4); // true
isEven.or(isPositive).test(-3); // false
isEven.negate().test(3);        // true

Function<Integer, Integer> doubleIt = n → n * 2;
len.andThen(doubleIt).apply("Java"); // 8 (len first, then doubleIt)
```

Variants

- **Bi-variants** (two inputs): `BiPredicate<T,U>`, `BiFunction<T,U,R>`, `BiConsumer<T,U>`. There is **no** `BiSupplier` – a method cannot return two values.
- **Operators**: `UnaryOperator<T>` (= `Function<T,T>`), `BinaryOperator<T>` (= `BiFunction<T,T,T>`, used by `reduce`).
- **Primitive specializations** avoid boxing overhead: `IntPredicate`, `IntFunction<R>`, `ToIntFunction<T>`, `IntSupplier`, `IntConsumer`, `DoubleFunction`, `LongBinaryOperator` ...

Comparator as a functional interface

```
List<String> trainees = new ArrayList<>(List.of("Kiran", "Asha", "Ravi"));

// Explicit lambda
trainees.sort((a, b) → a.compareTo(b));

// Method reference - same thing
trainees.sort(String::compareTo);

// TreeSet with a lambda comparator
TreeSet<Employee> byName = new TreeSet<>((e1, e2) → e1.name.compareTo(e2.name));

// Comparator factory methods - compose orders
list.sort(Comparator.comparing((Employee e) → e.name)
                    .thenComparingInt(e → e.age)
                    .reversed());
```

Method references

Shorthand for a lambda that *only* calls one existing method.

Form	Syntax	Equivalent lambda
Static method	<code>Integer::parseInt</code>	<code>s → Integer.parseInt(s)</code>
Instance method of a particular object	<code>System.out::println</code>	<code>x → System.out.println(x)</code>
Instance method of an arbitrary object of the type	<code>String::toUpperCase</code>	<code>s → s.toUpperCase()</code>
Constructor	<code>ArrayList::new</code>	<code>() → new ArrayList<>()</code>

```
trainees.forEach(System.out::println);  
List<Integer> lengths = trainees.stream().map(String::length).toList();
```

Static imports

```
import static org.pankaj.Date.display;      // one member  
import static java.lang.Math.*;            // all public static members  
  
display();                                 // no class prefix  
double h = sqrt(pow(3, 2) + pow(4, 2));    // instead of Math.sqrt(Math.pow(...))
```

- Only **public static** members can be statically imported.
- Two static imports with the **same member name** → *ambiguity* compile error when used.
- Use sparingly – over-use hides where a method comes from.

Records (Java 14 preview, standard 16)

A **record** is a concise, **immutable data carrier**. You declare the components; the compiler generates the rest.

```
record Data(String name, String address, int age, boolean gender) { }  
  
// Compiler auto-generates:  
// • canonical constructor Data(String, String, int, boolean)  
// • accessors          name(), address(), age(), gender() - NOT getName()  
// • toString()         Data[name=test, address=Mumbai, age=12, gender=true]  
// • equals() / hashCode() based on ALL components  
  
Data d = new Data("test", "Mumbai", 12, true);  
System.out.println(d.name());           // test  
System.out.println(d);                  // Data[name=test, ... ]
```

- Fields are **private final** → the object is **immutable** (no setters).

- **Cannot extend a class** (implicitly extends `java.lang.Record`), but **can implement interfaces**.
- You may add **static methods**, **instance methods**, and a **compact constructor** for validation:

```
record Range(int low, int high) {
    Range {                                     // compact constructor - runs before
field assignment
        if (low > high) throw new IllegalArgumentException("low > high");
    }
    int length() { return high - low; }          // extra method
}
```

Stream API

Day 14–15. Declarative data processing: build a pipeline, let the library iterate. Plus the helpers that surround it – `Optional`, enums, `hashCode/equals`, parallel streams, `Scanner`.

Core concepts

SOURCE	INTERMEDIATE OPS (lazy, return Stream)	TERMINAL OP (triggers everything)
<code>list.stream()</code>	<code>filter(...)</code> → <code>map(...)</code> → <code>sorted(...)</code>	<code>collect(...)</code> / <code>forEach(...)</code> / <code>count()</code>

- **Pipeline** = source → zero or more **intermediate** operations → exactly one **terminal** operation.
- **Lazy**: intermediate ops do *nothing* until a terminal op is called; then elements flow through *one at a time* (filter→map→... for element 1, then element 2...).
- Streams **do not store data** (they are a view), **do not mutate the source**, and are **single-use**:

```
Stream<String> s = list.stream();
s.forEach(System.out::println); // OK - consumed
s.count();                     // IllegalStateException: stream has already been
operated upon or closed
```

- **Sources**: `collection.stream()`, `Arrays.stream(arr)`, `Stream.of(a, b, c)`, `IntStream.range(0, 10)`, `Files.lines(path)`, `map.entrySet().stream()`. Arrays have no `.stream()` method – use `Arrays.stream(arr)` or convert to a `List`.

Setting: the Book model used below

```
enum BookGenre { TECHNOLOGY, FICTION, HISTORY }

class Book {
    private int bookId; private String bookName; private BookGenre bookGenre; private int
totalQuantity;
    Book(int id, String name, BookGenre g, int qty) { bookId = id; bookName = name;
bookGenre = g; totalQuantity = qty; }
    public int getBookId() { return bookId; }
    public String getBookName() { return bookName; }
    public BookGenre getBookGenre() { return bookGenre; }
    public int getTotalQuantity() { return totalQuantity; }
    @Override public String toString() { return bookId + ":" + bookName + ":" + bookGenre +
":" + totalQuantity; }
}
List<Book> bookList = List.of(
    new Book(1, "Java in Depth", BookGenre.TECHNOLOGY, 120),
    new Book(2, "Kafka on the Shore", BookGenre.FICTION, 40),
    new Book(3, "Kernel Internals", BookGenre.TECHNOLOGY, 15),
    new Book(4, "Sapiens", BookGenre.HISTORY, 200));
```

Intermediate operations

Operation	Takes	Does
<code>filter(Predicate)</code>	$T \rightarrow \text{boolean}$	Keeps elements where predicate is true
<code>map(Function)</code>	$T \rightarrow R$	Transforms each element ($1 \rightarrow 1$)
<code>flatMap(Function)</code>	$T \rightarrow$ <code>Stream<R></code>	Flattens nested structures ($1 \rightarrow \text{many}$)
<code>sorted()</code> / <code>sorted(Comparator)</code>	$- / (T, T) \rightarrow \text{int}$	Sorts (stateful – needs all elements)
<code>distinct()</code>	$-$	Removes duplicates using <code>hashCode()</code> / <code>equals()</code>
<code>limit(n)</code> / <code>skip(n)</code>	long	Take first n / drop first n
<code>peek(Consumer)</code>	$T \rightarrow \text{void}$	Debug hook – see elements pass by
<code>mapToInt</code> / <code>mapToDouble</code>	$T \rightarrow \text{primitive}$	Switch to <code>IntStream</code> (gives <code>sum()</code> , <code>average()</code> , <code>max()</code>)

```

List<String> trainees = List.of("Kiran", "Asha", "Karan", "Ravi", "Kiran");

trainees.stream()
    .filter(s → s.startsWith("K"))           // Predicate → Kiran, Karan, Kiran
    .map(String::toUpperCase)                // Function → KIRAN, KARAN, KIRAN
    .distinct()                             // → KIRAN, KARAN
    .sorted(Comparator.reverseOrder())      // → KIRAN, KARAN
    .limit(3)
    .forEach(System.out::println);          // terminal

int totalQty = bookList.stream().mapToInt(Book::getTotalQuantity).sum();           // 375
double avg    = bookList.stream().mapToInt(Book::getTotalQuantity).average().orElse(0);

```

Terminal operations

Operation	Returns	Note
<code>forEach(Consumer)</code>	void	Consume once; no result to reuse
<code>collect(Collector)</code>	List / Set / Map ...	Reusable result – the workhorse
<code>toList()</code>	<code>List<T></code>	Java 16+, unmodifiable
<code>count()</code>	long	
<code>min / max(Comparator)</code>	<code>Optional<T></code>	Empty stream → empty Optional
<code>anyMatch / allMatch / noneMatch(Predicate)</code>	boolean	Short-circuit
<code>findFirst() / findAny()</code>	<code>Optional<T></code>	Short-circuit
<code>reduce(identity, BinaryOperator)</code>	T	Fold to one value

```

// collect vs forEach: collect gives a reusable result
List<Book> tech = bookList.stream()
    .filter(b → b.getBookGenre() == BookGenre.TECHNOLOGY)
    .collect(Collectors.toList()); // or .toList() on Java 16+

// max with a comparator → Optional (could be empty)
Optional<Book> max = bookList.stream()
    .max(Comparator.comparingInt(Book::getTotalQuantity));
if (max.isPresent()) System.out.println(max.get()); // 4:Sapiens:HISTORY:200

boolean all = bookList.stream().allMatch(b → b.getTotalQuantity() > 20); // false
// (Kernel = 15)
boolean any = bookList.stream().anyMatch(b → b.getTotalQuantity() > 20); // true
boolean none = bookList.stream().noneMatch(b → b.getTotalQuantity() < 0); // true
long techCount = bookList.stream().filter(b → b.getBookGenre() ==
BookGenre.TECHNOLOGY).count(); // 2

int sumQty = bookList.stream().map(Book::getTotalQuantity).reduce(0, Integer::sum); //
375

```

Collectors

```

import java.util.stream.Collectors;

// groupingBy → Map<Key, List<T>> (one group per distinct key)
Map<BookGenre, List<Book>> byGenre = bookList.stream()
    .collect(Collectors.groupingBy(Book::getBookGenre));
// {TECHNOLOGY=[1: ... , 3: ... ], FICTION=[2: ... ], HISTORY=[4: ... ]}

// groupingBy with a downstream collector
Map<BookGenre, Long> countByGenre = bookList.stream()
    .collect(Collectors.groupingBy(Book::getBookGenre, Collectors.counting()));
Map<BookGenre, Integer> qtyByGenre = bookList.stream()
    .collect(Collectors.groupingBy(Book::getBookGenre,
Collectors.summingInt(Book::getTotalQuantity)));

// partitioningBy → exactly TWO groups: Map<Boolean, List<T>>
Map<Boolean, List<Book>> part = bookList.stream()
    .collect(Collectors.partitioningBy(b → b.getTotalQuantity() ≥ 100));
List<Book> above = part.get(true); // Java in Depth, Sapiens
List<Book> below = part.get(false); // Kafka, Kernel

// toMap → Map<K,V>
Map<Integer, String> idToName = bookList.stream()
    .collect(Collectors.toMap(Book::getBookId, Book::getBookName));

// joining → String
String names = bookList.stream().map(Book::getBookName).collect(Collectors.joining(", "));

```

	filter	partitioningBy	groupingBy
Result	Only the matching elements	Both groups (<code>true</code> / <code>false</code>)	N groups, one per key
Type	<code>Stream<T></code> → <code>collect</code>	<code>Map<Boolean, List<T>></code>	<code>Map<K, List<T>></code>

- `Collectors.toList()` returns an `ArrayList` (modifiable); `Stream.toList()` (Java 16+) returns an **unmodifiable** list; `toSet()` returns a `HashSet`.

Optional – a container that may be empty

Replaces returning `null` (and the resulting `NullPointerException`) with an explicit "maybe" type.

```
Optional<Employee> opt = repository.findById(101);           // returns Optional, never null

if (opt.isPresent()) {                                       // 1. explicit check
    Employee e = opt.get();                                   //   get() on empty →
    NoSuchElementException
}
if (opt.isEmpty()) System.out.println("not found");          // Java 11

Employee e1 = opt.orElse(new Employee("Guest"));            // 2. fallback value
Employee e2 = opt.orElseGet(() → loadDefault());             // 3. lazy fallback
Employee e3 = opt.orElseThrow(() → new RuntimeException("Not found")); // 4. throw
opt.ifPresent(emp → System.out.println(emp.getName()));      // 5. consume if present
String name = opt.map(Employee::getName).orElse("N/A");      // 6. transform

Optional<String> empty = Optional.empty();
Optional<String> some  = Optional.of("x");                   // of(null) → NPE
Optional<String> maybe = Optional.ofNullable(possiblyNull);
```

Enums

- A **type-safe fixed set of constants**. Each constant is a **singleton object** of the enum type.
- Implicitly `extends java.lang.Enum` → an enum **cannot extend a class** but **can implement interfaces**.
- Constants must be declared **first**; may have **fields, constructors (always private), methods**.

```

public enum BookGenre {
    TECHNOLOGY(2025), FICTION(1987), HISTORY(2011);    // constants FIRST, each calls the
// constructor

    private final int releaseYear;

    BookGenre(int year) { this.releaseYear = year; }    // implicitly private
    public int getReleaseYear() { return releaseYear; }
}

BookGenre g = BookGenre.FICTION;
g.name();    // "FICTION"
g.ordinal();    // 1 (declaration index)
g.getReleaseYear();    // 1987
BookGenre.values();    // BookGenre[] - all constants in order
BookGenre.valueOf("HISTORY");    // HISTORY (IllegalArgumentException if no such name)
g == BookGenre.FICTION;    // true - singletons, so == is safe

switch (g) {    // works directly with enums (no BookGenre. prefix in
// cases)
    case TECHNOLOGY → System.out.println("Tech");
    case FICTION, HISTORY → System.out.println("Reading");
}

```

hashCode() and equals() – required for distinct(), HashSet, HashMap

- Both come from `Object`; **override them together**. Hash-based structures check `hashCode()` **first** (fast bucket lookup), then `equals()` to confirm. Overriding only one breaks `HashSet`, `HashMap` keys and `distinct()`.
- Contract:** if `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` **must** hold. (The reverse is not required – collisions are fine.)

```
import java.util.Objects;

class Book {
    private int bookId;           // treat bookId as the primary key
    private String bookName;

    @Override
    public int hashCode() {
        return Objects.hash(bookId);           // same id → same hash
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Book)) return false; // also rejects null
        Book other = (Book) obj;
        return bookId == other.bookId;           // equality by id only
    }
}

bookList.stream().distinct().forEach(System.out::println);
// Two Books with the same bookId are now duplicates - even if bookName differs
```

Parallel streams

```
bookList.parallelStream()           // or .stream().parallel()
    .filter(b → b.getBookGenre() == BookGenre.TECHNOLOGY)
    .forEach(System.out::println);   // order NOT guaranteed
```

- Splits the data across the **common ForkJoinPool** threads (one per core) and handles synchronization for you.
- Worth it for **large datasets** with **CPU-heavy, independent** operations on multi-core machines. For **small** collections the thread coordination overhead makes it **slower** than a sequential stream.
- Never mutate shared state inside a parallel stream; use `forEachOrdered` if order matters.

Scanner – modern console input

```
import java.util.Scanner;

Scanner scanner = new Scanner(System.in);

String name = scanner.nextLine(); // full line including spaces → "Harry Potter"
String token = scanner.next();    // one token - stops at whitespace → "Harry"
int year = scanner.nextInt();     // parses an int (InputMismatchException on bad input)
double price = scanner.nextDouble();
boolean flag = scanner.nextBoolean();

scanner.close();                  // it's a resource - close it (closes System.in too)
```

The `nextInt()` + `nextLine()` trap: `nextInt()` leaves the newline in the buffer, so a following `nextLine()` returns `" "`. Fix: call `scanner.nextLine()` once after `nextInt()` to consume the leftover, or read everything with `nextLine()` and parse.

Generics and Wildcards

Day 15. Bounding a type parameter so a generic class or method accepts *some* types but not others.

	Object		
children"	Base	? extends Base → { Base, Derived, ... }	"current +
parents"		? super Base → { Base, Object }	"current +
	Derived	? super Derived → { Derived, Base, Object }	

Upper bounded – extends

- `T extends Base` : `T` may be **Base itself or any subclass**. Inside the class you may safely call `Base`'s methods on `T`.

```
class Base { void show() { System.out.println("Base"); } }
class Derived extends Base { }

class Data<T extends Base> {           // upper bound
    T value;
    void run() { value.show(); }       // legal - T is at least a Base
}

Data<Base> d1 = new Data<>();           // OK
Data<Derived> d2 = new Data<>();        // OK
// Data<Object> d3;                     // compile error - Object is not a Base
// Data<String> d4;                     // compile error - unrelated

static double sum(List<? extends Number> nums) { // accepts List<Integer>,
    List<Double>, ...
    double s = 0;
    for (Number n : nums) s += n.doubleValue();
    return s;
}
```

Lower bounded – `super`

- `? super Base` : the type may be `Base` itself or any parent (up to `Object`). You can safely **write** a `Base` into it.

```
static void addDerived(List<? super Derived> list) {    // List<Derived>, List<Base>,
List<Object>
    list.add(new Derived());                          // safe - every accepted list can
hold a Derived
}

// Hierarchy: Object → Base → Derived
// ? super Derived accepts: Derived, Base, Object
// ? super Base    accepts: Base, Object
```

Unbounded – `?`

```
static void printAll(List<?> list) {                // any List; you can only READ as Object
    for (Object o : list) System.out.println(o);
}
```

PECS – Producer `extends` , Consumer `super`

- Use `? extends T` when you only **read** (the collection *produces* T's for you).
- Use `? super T` when you only **write** (the collection *consumes* T's).
- The JDK follows this everywhere:

```
Collectors.partitioningBy(Predicate<? super T> predicate)    // predicate
CONSUMES T → super
Collectors.groupingBy(Function<? super T, ? extends K> classifier) // consumes T,
produces K
Collections.copy(List<? super T> dest, List<? extends T> src) // dest consumes,
src produces
```

Memory rule: `extends` = current class + **children**; `super` = current class + **parents**.

Multithreading and Synchronization

Day 15 – the big final topic. Running several paths of execution at once, and stopping them from corrupting each other's data.

Why threads

- Default execution is **synchronous** – top to bottom, one statement at a time. A method with an infinite loop (`while (true)`) **blocks everything after it** forever.
- **Thread (memorize):** *an independent path of execution that is given a separate time slice by the CPU.*
- A **single CPU** creates the *illusion* of simultaneity through **time slicing** – the scheduler gives each ready thread a short turn (≈ 20 ms) in **round-robin** fashion. On multi-core CPUs threads may truly run in parallel.
- Every Java program already has one thread: `main` .

Process vs thread

	Process	Thread
Memory	Independent address space	Shares the parent process's heap (own stack only)
Termination	Independent – one crashing does not kill another	Parent process dies → all its threads die
Creation cost	High (new address space, OS resources)	Low
Communication	IPC (pipes, sockets) – slow	Shared variables – fast, but needs synchronization
Example	Chrome and Word running together	Word's typing, spell-check and autosave

Three ways to create a thread

1. Extend `Thread`

```
class IncrementThread extends Thread {
    @Override
    public void run() { // the thread's body
        while (true) {
            System.out.println("Increment");
            try { Thread.sleep(100); } catch (InterruptedException e) {
e.printStackTrace(); }
        }
    }
}

IncrementThread inc = new IncrementThread();
inc.start(); // allocates thread resources, asks scheduler for a time slice, then JVM
calls run()
```

Critical trap: calling `inc.run()` directly is a **normal synchronous method call** – no new thread is created and `main` blocks inside the loop. Always call `start()` .

- Thread **life = life of** `run()` – when `run()` returns (or throws), the thread **dies**.
- You **cannot** `start()` **the same** `Thread` **object twice** → `IllegalThreadStateException` . Create a new `Thread` .

2. Implement `Runnable` (preferred)

```
class DecrementThread implements Runnable {           // Runnable = the TASK
    @Override
    public void run() { /* work */ }
}

Thread dec = new Thread(new DecrementThread());      // Thread = the RUNNER
dec.start();
```

Why prefer `Runnable` :

1. Extending `Thread` uses up your **single class inheritance** – you can't also extend `Employee` .
2. You are **not changing how** `Thread` **works**, only *what it runs* – so inheritance is semantically wrong; composition is right.
3. Same result, and the task can be handed to **thread pools** (`ExecutorService`).

3. Lambda (`Runnable` is a functional interface)

```
Runnable show = () → {
    while (true) {
        System.out.println("Show");
        try { Thread.sleep(100); } catch (InterruptedException e) { }
    }
};
new Thread(show).start();

new Thread(() → System.out.println("inline task")).start(); // one-liner
```

`Thread.sleep(ms)`

- The current thread **voluntarily gives up the CPU** for at least `ms` milliseconds – *"I have nothing to do right now, give my slice to someone else."*
- **Static** – it always sleeps the *calling* thread; `otherThread.sleep()` does **not** sleep `otherThread` .
- Throws the **checked** `InterruptedException` (another thread called `interrupt()` on it) – **try-catch is mandatory**.

Thread lifecycle



Useful `Thread` methods: `getName()`, `setName()`, `currentThread()`, `isAlive()`, `getState()`, `getId()`, `interrupt()`, `isDaemon()`, `getPriority()`.

Worker (user) vs daemon threads

	Worker / user thread	Daemon thread
Default?	Yes (including <code>main</code>)	No – <code>setDaemon(true)</code>
JVM shutdown	JVM will not exit while any worker is alive	JVM exits as soon as all workers finish – daemons are killed mid-execution
Purpose	Real work the user is waiting for	Background/service work (GC, monitoring, autosave)

```
Thread decThread = new Thread(new DecrementThread());
decThread.setDaemon(true); // MUST be called BEFORE start() - else
                             IllegalArgumentException
decThread.start();
```

- **Analogy:** MS Word's spell-checker is a **daemon** – closing Word kills it instantly. A 30-second file save is a **worker** – Word waits for it before closing.
- The **garbage collector** is a daemon thread.

Stopping threads properly

- `stop()` is **deprecated** – kills the thread mid-instruction, like pulling the power cable. A half-written file or half-updated data structure is left corrupt; locks may never be released.
- `System.exit()` – also abrupt; skips all cleanup and `finally` blocks of *every* thread. Avoid in production code.
- **Correct approach** – a **shared flag for graceful shutdown**:

```

class Worker implements Runnable {
    volatile boolean runningStatus = true;    // volatile: writes visible to all threads
    immediately

    @Override
    public void run() {
        while (runningStatus) {                // controlled loop
            doOneUnitOfWork();
            try { Thread.sleep(25); } catch (InterruptedException e) { runningStatus =
false; }
        }
        cleanup();                            // run() returns naturally → clean
termination
    }
    void stopGracefully() { runningStatus = false; }
}

Worker w = new Worker();
new Thread(w).start();
// later, e.g. on window close:
w.stopGracefully();

```

- Alternative idiom: `thread.interrupt()` + checking `Thread.currentThread().isInterrupted()` inside the loop.

Priority

- `setPriority(1-10)`; constants `Thread.MIN_PRIORITY = 1`, `NORM_PRIORITY = 5` (default), `MAX_PRIORITY = 10`.
- Priority is only a **request/hint** to the scheduler – the JVM/OS **may ignore it** (Windows maps 10 levels to fewer OS levels). Never build correctness on priorities.

Context switching and performance

- Modern OSes use **preemptive scheduling** – the OS *takes* the CPU away when a time slice ends; no thread can monopolize it.
- **Context switch**: when a thread's slice ends, the OS **saves its context** (program counter = last instruction, register values, stack pointer) and **restores** it on the thread's next turn. This costs time.
- **Performance vs throughput trade-off**:
 - Each *individual* task gets **slower** (switch overhead: a 2000 ms job may take 2200 ms).
 - But *total work* done in a period **increases** – download a file *while* writing a letter, instead of waiting.

Synchronization – the race condition

Bank account example: balance ₹10,000; a deposit of ₹5,000 and a withdrawal of ₹5,000 run **at the same time**.

```

// Thread 1 - Deposit                                // Thread 2 - Withdrawal
int balance = getBalance(7774);    // 10,000
balance = balance + 5000;           // 15,000 (in T1's local variable only)
// ← CONTEXT SWITCH before setBalance runs

(stale!)

5,000
setBalance(7774, 15000);           // overwrites → 15,000 ... or the other order → 5,000
// Correct answer 10,000 is never produced. Money is created or lost.

int balance = getBalance(7774);    // 10,000

balance = balance - 5000;           // 5,000
setBalance(7774, balance);         // writes

```

- A **race condition** = the result depends on the *timing* of thread switches. The sequence *read* → *modify* → *write* must not be interrupted – it is a **critical section** that needs to be **atomic**.
- Hand-rolled locking with a `boolean isLocked` flag is dangerous: the check-and-set itself is not atomic, and forgetting to unlock causes **starvation/deadlock**. Java provides **synchronized**.

The **synchronized** keyword

Every Java object has an intrinsic **monitor lock**. A thread must **acquire** the lock to enter a **synchronized** region; others **block** until it is released (automatically on exit, even via exception).

```

// 1. Synchronized BLOCK - lock only the critical section (preferred: minimal locking)
public void display() {
    // non-critical statements - all threads run these freely
    synchronized (sharedObject) {           // lock on a specific object
        // ONLY ONE thread inside at a time
    }
    // more free-running statements
}

// 2. Synchronized METHOD - whole body is critical; lock = 'this' (or the Class object if static)
public synchronized void deposit(double amount) {
    balance += amount;
}

```

- `Vector` / `Hashtable` / `StringBuffer` = **all methods synchronized** (safe, slower).
- `ArrayList` / `HashMap` / `StringBuilder` = **not** synchronized (faster, single-threaded use).
- Lock on the **shared data object**, not on a new object each time (`synchronized (new Object())` locks nothing).

Shared-data example with two threads

```
class Data { int num = 0; }

class IncrementThread extends Thread {
    private final Data objData;
    IncrementThread(Data d) { this.objData = d; }
    @Override public void run() {
        while (true) {
            synchronized (objData) { // same lock object as Decrement
                objData.num++;
                System.out.println("Increment: " + objData.num);
            }
            try { Thread.sleep(100); } catch (InterruptedException e) { }
        }
    }
}

class DecrementThread extends Thread {
    private final Data objData;
    DecrementThread(Data d) { this.objData = d; }
    @Override public void run() {
        while (true) {
            synchronized (objData) {
                objData.num--;
                System.out.println("Decrement: " + objData.num);
            }
            try { Thread.sleep(100); } catch (InterruptedException e) { }
        }
    }
}

Data shared = new Data();
new IncrementThread(shared).start();
new DecrementThread(shared).start();
// Without synchronized: num jumps inconsistently (-1, -2, -1, 1 ...) - prints don't match values
// With synchronized:    clean alternation 1 → 0 → 1 → 0 (or 0 → -1 → 0)
```

Inter-thread communication: `wait()` / `notify()` / `notifyAll()`

- Defined on `Object` (every object is a potential monitor), `final`, and **must be called inside a `synchronized` block on the same object** – else `IllegalMonitorStateException`.
- `wait()` releases the lock and pauses until notified (or timeout). `notify()` wakes **one** waiting thread (typically the first – FIFO-ish). `notifyAll()` wakes **all**; the first to re-acquire the lock proceeds.

```

Object dummy = new Object();

// Waiting thread
synchronized (dummy) {
    while (!conditionMet)           // always re-check in a loop (spurious wake-ups)
        dummy.wait(5000);           // release lock, pause up to 5 s; resumes early if
    notified
}

// Signalling thread
synchronized (dummy) {
    conditionMet = true;
    dummy.notify();                  // wake FIRST waiting thread
    // dummy.notifyAll();           // wake ALL waiting threads
}

```

	<code>sleep(ms)</code>	<code>wait(ms)</code>
Defined in	<code>Thread</code> (static)	<code>Object</code>
Releases lock?	No – holds any locks it has	Yes – releases the monitor
Needs <code>synchronized</code> ?	No	Yes
Wakes up by	Timeout / <code>interrupt()</code>	<code>notify()</code> / <code>notifyAll()</code> / timeout

`join()` – wait for another thread to finish

```

Thread rectangleThread = new Thread(() → computeArea());
rectangleThread.start();

rectangleThread.join();           // current thread waits until rectangleThread TERMINATES
rectangleThread.join(3000);       // wait at most 3 s, then proceed anyway
System.out.println("Area ready (or timed out)");

```

- Typical use: `main` starts N worker threads, `join()` s each, then combines results.
- Throws `InterruptedException` – needs try-catch.

Quick reference: the multithreading trap list

1. `run()` vs `start()` – direct `run()` = no new thread.
2. `setDaemon(true)` must precede `start()`.
3. Same `Thread` object cannot be started twice.
4. `wait/notify` outside `synchronized` → `IllegalMonitorStateException`.
5. `sleep` keeps locks; `wait` releases them.
6. Priorities are hints; `stop()` is deprecated; use a `volatile` flag.
7. Lock on the *shared* object, and use the *same* lock in every thread that touches the data.

Date and Time API

Day 15. Legacy `java.util.Date` vs the Java 8 `java.time` package.

Legacy

```
import java.util.Date;

Date currentDate = new Date();           // Wed Sep 23 19:40:14 IST 2026
long millis = System.currentTimeMillis(); // ms since 1 Jan 1970 00:00 UTC (Unix epoch)
Date fromMillis = new Date(millis);
```

- `java.util.Date` is **mutable**, not thread-safe, month is 0-based, most methods deprecated. Use it only when a legacy API demands it.

Java 8 `java.time` – immutable and time-zone aware

Class	Holds	Example
<code>LocalDate</code>	date only	<code>2026-09-23</code>
<code>LocalTime</code>	time only	<code>19:46:30</code>
<code>LocalDateTime</code>	date + time, no zone	<code>2026-09-23T19:46</code>
<code>ZonedDateTime</code>	date + time + zone	<code>2026-09-23T19:46+05:30[Asia/Kolkata]</code>
<code>Instant</code>	machine timestamp (UTC)	<code>2026-09-23T14:16:00Z</code>
<code>Duration</code> / <code>Period</code>	time-based / date-based amount	<code>PT2H</code> , <code>P1Y2M</code>

```

import java.time.*;
import java.time.format.DateTimeFormatter;

LocalDateTime india = LocalDateTime.now(ZoneId.of("Asia/Kolkata"));
LocalDateTime chicago = LocalDateTime.now(ZoneId.of("America/Chicago")); // applies UTC
// offset per zone
LocalDateTime specific = LocalDateTime.of(2026, 9, 23, 19, 46); // Y, M (1-
// based!), D, H, Min

LocalDate today = LocalDate.now();
LocalDate due = today.plusDays(30).plusMonths(1); // immutable - returns a NEW object
today.getDayOfWeek(); // WEDNESDAY
today.isLeapYear();
today.lengthOfMonth();

// Localization - same instant, different display
DateTimeFormatter indiaFmt = DateTimeFormatter.ofPattern("dd/MM/yyyy"); // 23/09/2026
DateTimeFormatter usaFmt = DateTimeFormatter.ofPattern("MM/dd/yyyy"); // 09/23/2026
System.out.println(today.format(indiaFmt));
LocalDate parsed = LocalDate.parse("23/09/2026", indiaFmt);

// Differences
Period p = Period.between(LocalDate.of(2026, 1, 1), today); // years/months/days
Duration d = Duration.between(LocalTime.of(9, 0), LocalTime.now());

```

- Every `java.time` object is **immutable and thread-safe**; all "modifying" methods return a new instance (compare `String`).
 - Prefer `LocalDate` / `LocalDateTime` for business dates, `Instant` for timestamps, `ZonedDateTime` when the zone matters (scheduling across regions).
-

Interview Traps and Final Checklist

Java version cheat-sheet

Version	Year	Features in this course
Java 2 (1.2)	1998	JIT compiler, Collections Framework, Swing
Java 5	2004	Generics, enums, auto-boxing, varargs, enhanced <code>for</code> , static import
Java 7	2011	Diamond <code><></code> , try-with-resources, multi-catch, <code>String</code> in <code>switch</code>
Java 8	2014	Lambdas, functional interfaces, Stream API, default/static interface methods, <code>Optional</code>, <code>java.time</code>
Java 9	2017	Private interface methods, modules, <code>List.of()</code> , <code>readAllBytes()</code> , G1 default
Java 14	2020	Switch expressions (standard), records (preview)
Java 16	2021	Records, pattern matching for <code>instanceof</code> , <code>Stream.toList()</code>
Java 17 (LTS)	2021	Sealed classes/interfaces
Java 21 (LTS)	2023	Pattern matching for <code>switch</code> , virtual threads, sequenced collections

Top interview traps

1. `run()` vs `start()` – direct `run()` is a plain method call; no new thread.
2. `==` vs `.equals()` on Strings; literals share a **String Pool** object, `new String()` does not.
3. **Collection (interface) vs Collections (utility class).**
4. **Comparable** (1 arg, inside the class, one natural order) vs **Comparator** (2 args, separate, many orders).
5. **Checked vs unchecked** exceptions; catch order **derived first**, base last.
6. **Lambda vs anonymous class** – 7 differences (Section 12 table).
7. **Streams are single-use** → `IllegalStateException` on reuse.
8. `System.gc()` is only a request; `finalize()` deprecated; no destructors in Java.
9. **Marker interface** (`Serializable` has no methods); `transient` skips a field.
10. **Overloading works via name mangling** (name + argument types) at compile time; **overriding via the VFT** at runtime.
11. `public void Data()` is a method, not a constructor.
12. **Defining any constructor removes the default one.**
13. `Map` is not a `Collection`.
14. `hashCode()` and `equals()` must be overridden together.
15. `super(...)` / `this(...)` must be the first statement in a constructor.

16. `static` methods are hidden, not overridden.
17. `final` reference $\neq$ immutable object.
18. `Integer` cache `-128...127` makes `==` lie for wrappers – use `.equals()`.
19. `setDaemon(true)` before `start()`; `wait()` needs `synchronized`.
20. Java is **pass-by-value** – the value copied for objects is the reference.

One-page mental map



Self-test: can you answer these in one line each?

- Why is a `.exe` not portable but a `.class` is?
- Order of execution for `new Derived()` when both classes have static blocks, init blocks and constructors?
- Why does `Employee e = new Manager()` compile, but `e.getHRA()` does not?
- What happens if `catch (Exception e)` appears before `catch (IOException e)`?
- What does `transient` do on read-back?
- Why does `TreeSet` reject an object whose class lacks `compareTo`?
- Why did Java 8 need `default` methods?
- What is wrong with `while (!done) { }` if `done` is not `volatile`?
- What does `Collectors.partitioningBy` return that `filter` cannot?
- Give the `? extends` vs `? super` one-line memory rule.

Coverage note: these notes rebuild and expand the original *OOPJ Complete Revision Notes* across all 15 sessions (Day 1 – Day 15, 7–23 Sep 2026). Every topic now carries a runnable code sample, a comparison table where two things are commonly confused, and the exam-style definition where one exists.